



University of Dhaka

Department of Computer Science and Engineering

CSE2211: Database Management Systems-1

Lab Project

VoyageVista: Travel Booking Platform

A Comprehensive Database Management System

Lab Group: A (ODD)

H.M. Mehedi Hasan (Roll: 13)

MD. Abu Bakar Siddique (Roll: 47)

Submitted To:

Mr. Abu Ahmed Ferdaus, Dept. of CSE, University of Dhaka

Mr. Redwan Ahmed Rizvee, Dept of CSE, University of Dhaka

Submission Date: July 17, 2025

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	System Objectives	4
1.3	Technology Stack	4
2	Database Design	4
2.1	Database Schema Overview	4
2.2	Schema Diagram	5
2.3	Entity-Relationship Diagram	6
2.4	Entity Descriptions	6
2.4.1	Users Table	6
2.4.2	Admins Table	6
2.4.3	Destinations Table	6
2.4.4	Packages Table	6
2.4.5	Package Detail Schedule Table	7
2.4.6	Package Images Table	7
2.4.7	Bookings Table	7
2.4.8	Promo Codes Table	7
2.4.9	Blogs Table	7
2.5	Database Constraints	7
2.5.1	Primary Key Constraints	7
2.5.2	Foreign Key Constraints	8
2.5.3	Unique Constraints	8
2.5.4	Check Constraints	8
3	Database Implementation	9
3.1	DDL Statements	9
3.1.1	Users Table Creation	9
3.1.2	Admins Table Creation	9
3.1.3	Destinations Table Creation	9
3.1.4	Packages Table Creation	10
3.1.5	Bookings Table Creation	10
3.1.6	Blogs Table Creation	11
3.1.7	Promo Codes Table Creation	11
3.1.8	Package Detail Schedule Table Creation	12
3.1.9	Package Images Table Creation	12
3.2	Populated Table Previews	13
3.2.1	admins Table Snapshot	13
3.2.2	users Table Snapshot	13
3.2.3	packages Table Snapshot	13
3.2.4	package-detail-schedule Table Snapshot	13
3.2.5	package-images Table Snapshot	13
3.2.6	bookings Table Snapshot	14
3.2.7	destinations Table Snapshot	14
3.2.8	promo-codes Table Snapshot	14

3.2.9	blogs Table Snapshot	14
4	Query Examples	14
4.1	Basic Queries with Joins	14
4.1.1	Cross Product Example	14
4.1.2	Left Outer Join Example	15
4.1.3	Join with ON Example	15
4.2	Nested Subqueries	16
4.2.1	Subquery with ANY Example	16
4.2.2	Subquery with SOME Example	17
4.2.3	Subquery with ALL Example	17
4.2.4	Subquery with EXISTS Example	18
4.3	Subqueries in Different Clauses	18
4.3.1	Subquery in FROM Clause Example	18
4.3.2	Subquery in WHERE Clause Example	19
4.3.3	Subquery in SELECT Clause Example	20
4.4	Advanced Clauses	21
4.4.1	ORDER BY Clause Example	21
4.4.2	GROUP BY Clause Example	21
4.4.3	HAVING Clause Example	22
4.4.4	WITH Clause (CTE) Example	23
4.5	String and Set Operations	24
4.5.1	String Operations Example 4	24
4.6	Update and Delete Operations	24
4.6.1	Simple UPDATE Example	24
4.6.2	DELETE Query Example	25
4.7	Built-in Functions and Aggregates	25
4.7.1	Aggregate Functions Example	25
4.7.2	Date and Time Functions Example	26
5	Views Implementation	27
5.1	View Creation	27
5.1.1	CREATE VIEW with Joins and Aggregates Example	27
5.1.2	CREATE VIEW for User Activity Summary	28
5.2	Queries Using Views	28
5.2.1	Using package_summary View	28
5.2.2	Using user_activity_summary View	29
6	Functional Dependencies and Normalization	30
6.1	Functional Dependencies Analysis	30
6.1.1	Users Table Functional Dependencies	30
6.1.2	Destinations Table Functional Dependencies	31
6.1.3	Packages Table Functional Dependencies	31
6.1.4	Package Detail Schedule Table Functional Dependencies	31
6.1.5	Promo Codes Table Functional Dependencies	31
6.1.6	Bookings Table Functional Dependencies	32
6.1.7	Blogs Table Functional Dependencies	32
6.1.8	Admins Table Functional Dependencies	32
6.1.9	Package Images Table Functional Dependencies	32

6.2	Normalization Analysis	33
6.2.1	First Normal Form (1NF)	33
6.2.2	Second Normal Form (2NF)	33
6.2.3	Third Normal Form (3NF)	34
6.2.4	Boyce-Codd Normal Form (BCNF)	36
7	Frontend Design and Implementation	38
7.1	Frontend Architecture	38
7.1.1	Technology Stack	38
7.2	Frontend Architecture Advantages	38
7.2.1	Advantages	38
7.2.2	Disadvantages	38
7.3	Key Features Implementation	39
7.3.1	User Management	39
7.3.2	Package Management	39
7.3.3	Booking System	39
7.3.4	Admin Dashboard	39
7.4	API Integration	39
7.4.1	API Endpoints	39
7.5	Website Snapshots	41
8	Conclusion	45
8.1	Summary	45
8.2	Learning Outcomes	45
8.3	Future Enhancements	45

1 Introduction

1.1 Project Overview

VoyageVista represents a modern approach to **travel booking systems**, combining sophisticated database design with contemporary web technologies. The platform addresses the growing need for efficient travel management systems that can handle complex relationships between **users, travel packages, destinations, bookings, and promotional offers**.

1.2 System Objectives

The primary objectives of the VoyageVista system include:

- Efficient management of travel packages and destinations
- Seamless user registration and booking processes
- Comprehensive administrative controls and analytics
- Secure authentication and authorization mechanisms
- Scalable database architecture supporting concurrent operations
- Implementation of advanced database concepts and query optimization

1.3 Technology Stack

The system utilizes a modern technology stack:

- **Database:** PostgreSQL 15 with UUID primary keys
- **Backend:** FastAPI with SQLAlchemy and Alembic migrations
- **Frontend:** Next.js 15 with React 19 and TypeScript
- **Caching:** Redis for session management
- **Storage:** Supabase for file management
- **Authentication:** JWT with dual role system

2 Database Design

2.1 Database Schema Overview

The VoyageVista database consists of **9 main entities** designed to handle the complete travel booking ecosystem. The schema incorporates various data types, constraints, and relationships to ensure data integrity and efficient operations.

2.2 Schema Diagram

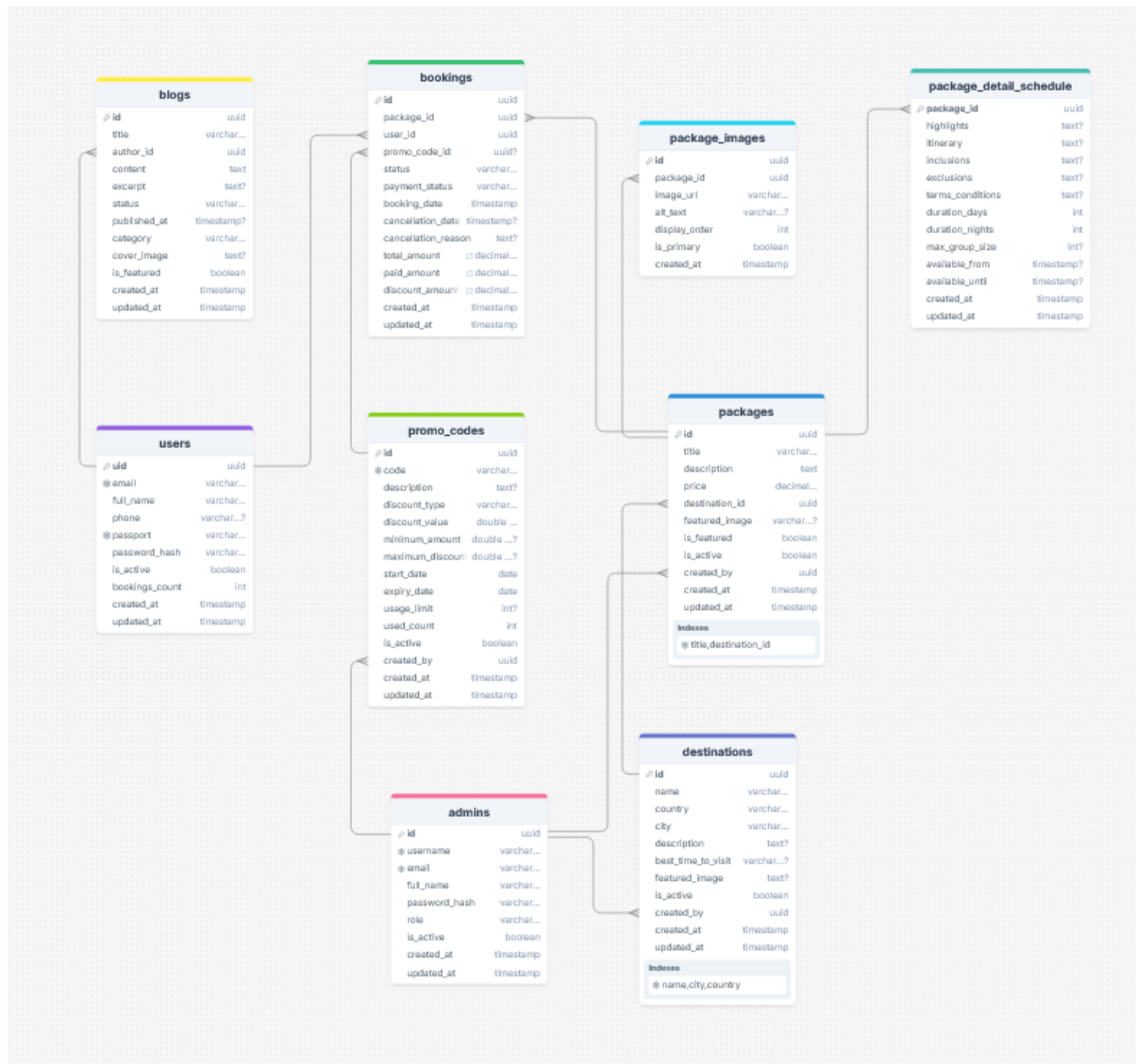


Figure 1: Schema Diagram

2.3 Entity-Relationship Diagram

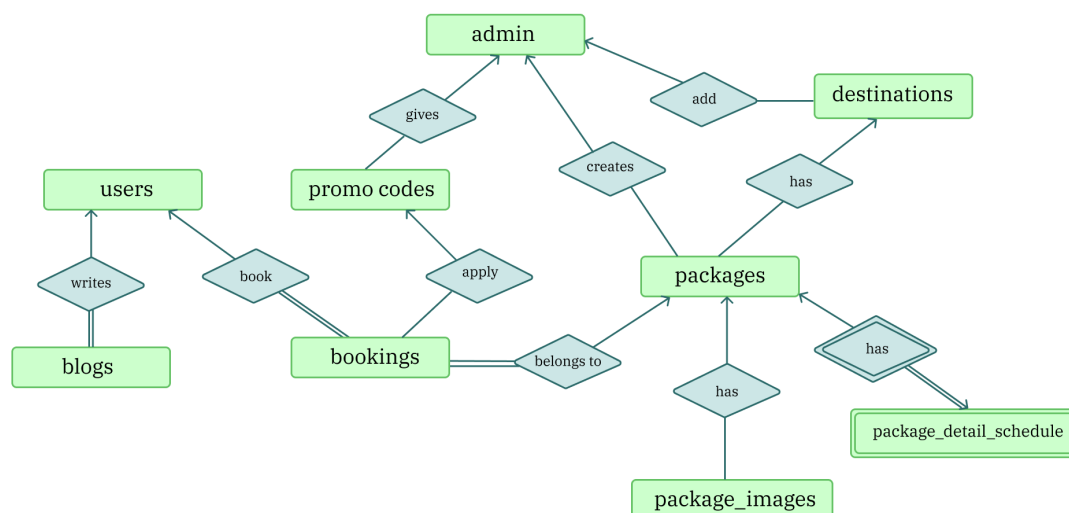


Figure 2: ER Diagram

2.4 Entity Descriptions

2.4.1 Users Table

Stores customer information with comprehensive profile data including authentication credentials, contact information, and passport details for international travel verification.

2.4.2 Admins Table

Manages administrative users with role-based access control, supporting different administrative levels and secure authentication mechanisms.

2.4.3 Destinations Table

Contains travel destination information including geographical data, descriptions, and metadata for package association and user browsing.

2.4.4 Packages Table

Central entity for travel packages, linking destinations with pricing, availability, and promotional features.

2.4.5 Package Detail Schedule Table

Extended package information including detailed itineraries, inclusions, exclusions, and scheduling constraints.

2.4.6 Package Images Table

Image management for packages with support for multiple images, ordering, and primary image designation.

2.4.7 Bookings Table

Transaction records for user bookings including payment tracking, status management, and promotional code integration.

2.4.8 Promo Codes Table

Promotional discount system with flexible discount types, usage limits, and expiration management.

2.4.9 Blogs Table

Content management system for travel-related articles, guides, and promotional content with publishing workflow.

2.5 Database Constraints

2.5.1 Primary Key Constraints

All tables implement UUID primary keys for enhanced security and distributed system compatibility:

- `users.uid` - Primary key for users table
- `admins.id` - Primary key for admins table
- `destinations.id` - Primary key for destinations table
- `packages.id` - Primary key for packages table
- `bookings.id` - Primary key for bookings table
- `promo_codes.id` - Primary key for promo codes table
- `blogs.id` - Primary key for blogs table

2.5.2 Foreign Key Constraints

Referential integrity maintained through foreign key relationships:

- `packages.destination_id` → `destinations.id`
- `packages.created_by` → `admins.id`
- `destinations.created_by` → `admins.id`
- `bookings.user_id` → `users.uid`
- `bookings.package_id` → `packages.id`
- `bookings.promo_code_id` → `promo_codes.id`
- `blogs.author_id` → `users.uid`
- `promo_codes.created_by` → `admins.id`
- `package_detail_schedule.package_id` → `packages.id` (CASCADE DELETE)
- `package_images.package_id` → `packages.id` (CASCADE DELETE)

2.5.3 Unique Constraints

Data uniqueness enforced at database level:

- `users.email` - Unique user email identification
- `users.passport` - Unique passport numbers
- `admins.username` - Unique admin usernames
- `admins.email` - Unique admin emails
- `promo_codes.code` - Unique promotional codes
- `destinations.name, city, country` - Unique destination location combination
- `packages.title, destination_id` - Unique package title per destination

2.5.4 Check Constraints

Business logic enforcement through check constraints:

- `packages.price >= 0` - Non-negative pricing
- `promo_codes.discount_value >= 0` - Valid discount values
- `bookings.total_amount >= 0` - Non-negative amounts
- `package_detail_schedule.duration_days >= 1` - Minimum duration

3 Database Implementation

3.1 DDL Statements

3.1.1 Users Table Creation

```
1 CREATE TABLE users (  
2     uid uuid NOT NULL,  
3     email character varying(255) NOT NULL,  
4     full_name character varying(255) NOT NULL,  
5     phone character varying(20),  
6     passport character varying(255) NOT NULL,  
7     password_hash character varying(255) NOT NULL,  
8     is_active boolean NOT NULL,  
9     bookings_count integer NOT NULL,  
10    created_at timestamp without time zone NOT NULL,  
11    updated_at timestamp without time zone NOT NULL,  
12  
13    CONSTRAINT unique_user_email UNIQUE (email),  
14    CONSTRAINT unique_user_passport UNIQUE (passport),  
15    CONSTRAINT users_pkey PRIMARY KEY (uid)  
16 );  
17  
18 SELECT * FROM users;
```

Listing 1: Users Table DDL with Data Query

3.1.2 Admins Table Creation

```
1 CREATE TABLE admins (  
2     id uuid NOT NULL,  
3     username character varying(50) NOT NULL,  
4     email character varying(255) NOT NULL,  
5     full_name character varying(255) NOT NULL,  
6     password_hash character varying(255) NOT NULL,  
7     role character varying(50) NOT NULL,  
8     is_active boolean NOT NULL,  
9     created_at timestamp without time zone NOT NULL,  
10    updated_at timestamp without time zone NOT NULL,  
11  
12    CONSTRAINT unique_admin_username UNIQUE (username),  
13    CONSTRAINT unique_admin_email UNIQUE (email),  
14    CONSTRAINT admins_pkey PRIMARY KEY (id)  
15 );  
16  
17 SELECT * FROM admins;
```

Listing 2: Admins Table DDL with Data Query

3.1.3 Destinations Table Creation

```
1 CREATE TABLE destinations (  
2     id uuid NOT NULL,  
3     name character varying(255) NOT NULL,
```

```

4  country character varying(100) NOT NULL,
5  city character varying(100) NOT NULL,
6  description text,
7  best_time_to_visit character varying(100),
8  featured_image text,
9  is_active boolean NOT NULL,
10 created_by uuid NOT NULL,
11 created_at timestamp NOT NULL,
12 updated_at timestamp NOT NULL,
13
14  CONSTRAINT unique_destination_name_location UNIQUE (name, city,
15  country),
16  CONSTRAINT destinations_created_by_fkey FOREIGN KEY (created_by)
17  REFERENCES admins(id),
18  CONSTRAINT destinations_pkey PRIMARY KEY (id)
19 );
20
21 SELECT * FROM destinations;

```

Listing 3: Destinations Table DDL with Data Query

3.1.4 Packages Table Creation

```

1  CREATE TABLE packages (
2      id uuid NOT NULL,
3      title character varying(255) NOT NULL,
4      description text NOT NULL,
5      price numeric(10,2) NOT NULL,
6      destination_id uuid NOT NULL,
7      featured_image character varying(500),
8      is_featured boolean NOT NULL,
9      is_active boolean NOT NULL,
10     created_by uuid NOT NULL,
11     created_at timestamp NOT NULL,
12     updated_at timestamp NOT NULL,
13
14     CONSTRAINT packages_destination_id_fkey FOREIGN KEY (destination_id
15     ) REFERENCES destinations(id),
16     CONSTRAINT packages_created_by_fkey FOREIGN KEY (created_by)
17     REFERENCES admins(id),
18     CONSTRAINT unique_title_per_destination UNIQUE (title,
19     destination_id),
20     CONSTRAINT packages_pkey PRIMARY KEY (id)
21 );
22
23 SELECT * FROM packages;

```

Listing 4: Packages Table DDL with Data Query

3.1.5 Bookings Table Creation

```

1  CREATE TABLE bookings (
2      id uuid NOT NULL,
3      package_id uuid NOT NULL,
4      user_id uuid NOT NULL,
5      promo_code_id uuid,

```

```

6      status character varying(20) NOT NULL,
7      payment_status character varying(20) NOT NULL,
8      booking_date timestamp without time zone NOT NULL,
9      cancellation_date timestamp without time zone,
10     cancellation_reason text,
11     total_amount numeric(10,2) DEFAULT 0.0 NOT NULL,
12     paid_amount numeric(10,2) DEFAULT 0.0 NOT NULL,
13     discount_amount numeric(10,2) DEFAULT 0.0 NOT NULL,
14     created_at timestamp without time zone NOT NULL,
15     updated_at timestamp without time zone NOT NULL,
16
17     CONSTRAINT bookings_package_id_fkey FOREIGN KEY (package_id)
18     REFERENCES packages(id),
19     CONSTRAINT bookings_user_id_fkey FOREIGN KEY (user_id) REFERENCES
20     users(uid),
21     CONSTRAINT bookings_promo_code_id_fkey FOREIGN KEY (promo_code_id)
22     REFERENCES promo_codes(id),
23     CONSTRAINT bookings_pkey PRIMARY KEY (id)
24 );
25
26 SELECT * FROM bookings;

```

Listing 5: Bookings Table DDL with Data Query

3.1.6 Blogs Table Creation

```

1 CREATE TABLE blogs (
2     id uuid NOT NULL PRIMARY KEY,
3     title character varying(255) NOT NULL,
4     author_id uuid NOT NULL,
5     content text NOT NULL,
6     excerpt text,
7     status character varying(20) NOT NULL,
8     published_at timestamp without time zone,
9     category character varying(50) NOT NULL,
10    cover_image text,
11    is_featured boolean NOT NULL,
12    created_at timestamp without time zone NOT NULL,
13    updated_at timestamp without time zone NOT NULL,
14
15    CONSTRAINT blogs_author_id_fkey
16    FOREIGN KEY (author_id) REFERENCES public.users(uid)
17 );

```

Listing 6: Blogs Table DDL

3.1.7 Promo Codes Table Creation

```

1 CREATE TABLE promo_codes (
2     id uuid NOT NULL PRIMARY KEY,
3     code character varying(50) NOT NULL,
4     description text,
5     discount_type character varying(20) NOT NULL,
6     discount_value double precision NOT NULL,
7     minimum_amount double precision,
8     maximum_discount double precision,

```



```

9      start_date date NOT NULL,
10     expiry_date date NOT NULL,
11     usage_limit integer,
12     used_count integer NOT NULL,
13     is_active boolean NOT NULL,
14     created_by uuid NOT NULL,
15     created_at timestamp NOT NULL,
16     updated_at timestamp NOT NULL,
17
18     CONSTRAINT promo_codes_code_key UNIQUE (code),
19     CONSTRAINT promo_codes_created_by_fkey
20     FOREIGN KEY (created_by) REFERENCES public.admins(id)
21 );

```

Listing 7: Promo Codes Table DDL

3.1.8 Package Detail Schedule Table Creation

```

1 CREATE TABLE package_detail_schedule (
2     package_id uuid NOT NULL PRIMARY KEY,
3     highlights text,
4     itinerary text,
5     inclusions text,
6     exclusions text,
7     terms_conditions text,
8     duration_days integer NOT NULL,
9     duration_nights integer NOT NULL,
10    max_group_size integer,
11    available_from timestamp without time zone,
12    available_until timestamp without time zone,
13    created_at timestamp without time zone NOT NULL,
14    updated_at timestamp without time zone NOT NULL,
15
16    CONSTRAINT package_detail_schedule_package_id_fkey
17    FOREIGN KEY (package_id) REFERENCES public.packages(id) ON DELETE
18    CASCADE
19 );

```

Listing 8: Package Detail Schedule Table DDL

3.1.9 Package Images Table Creation

```

1 CREATE TABLE package_images (
2     id uuid NOT NULL PRIMARY KEY,
3     package_id uuid NOT NULL,
4     image_url character varying(500) NOT NULL,
5     alt_text character varying(255),
6     display_order integer NOT NULL,
7     is_primary boolean NOT NULL,
8     created_at timestamp without time zone NOT NULL,
9
10    CONSTRAINT package_images_package_id_fkey
11    FOREIGN KEY (package_id) REFERENCES public.packages(id) ON DELETE
12    CASCADE
13 );

```

Listing 9: Package Images Table DDL

3.2 Populated Table Previews

3.2.1 admins Table Snapshot

	id [PK] uuid	username character varying (50)	email character varying (255)	full_name character varying (255)	password_hash character varying	role character varying (50)	is_active boolean	created_at timestamp without time zone	updated_at timestamp without time zone
1	929b7ad6-d9c5-4cf8-b...	Mehedi26696	hasanmehedi26696@gmail.com	H.M.Mehedi Hasan	\$2b\$12\$2SNe...	admin	true	2025-07-09 00:57:4...	2025-07-09 01:01:53.472271
2	3a24279f-89b4-49c5-8...	admin	admin@vistavoyage.com	System Administrator	\$2b\$12\$N1D...	super_admin	true	2025-07-08 21:02:0...	2025-07-13 22:52:59.375718
3	c792ac34-72ed-4cd0-9...	abs	bojackabs@gmail.com	ABS ABS	\$2b\$123/oue...	admin	true	2025-07-14 01:02:0...	2025-07-14 01:02:07.005797
4	123e4567-e89b-12d3-...	migration_admin	migration_admin@example.com	Migration Admin	dummyhash	admin	true	2025-07-15 22:30:1...	2025-07-15 22:30:18.118188
5	a1a1a1a1-a1a1-a1a1-a...	admin_john	john@travel.com	John Admin	hashed_pass...	superadmin	true	2025-07-16 01:32:1...	2025-07-16 01:32:18.642621

3.2.2 users Table Snapshot

	uid [PK] uuid	email character varying (255)	full_name character varying (255)	phone character varying (20)	passport character varying (255)	password_hash character varying	is_active boolean	bookings_co integer	created_at timestamp w	updated_at timestamp with
1	5a85f0a3-6c0b-4578-b6ec-7...	hasanmehedi26696@gmail.com	Mehedi Hasan	+8801319926696	12345678	\$2b\$12\$Chm...	true	0	2025-07-0...	2025-07-08 ...
2	144b6e69-8794-46d6-9a9c-...	jfweljw@gmail.com	New	+8801319926696	2375983	\$2b\$12\$MwI...	true	0	2025-07-1...	2025-07-15 ...
3	c361a0ec-603e-482a-b52e-7...	bojackabs@gmail.com	Abu Bakar Siddique	+880131992657	5789273982	\$2b\$12\$SLbv...	true	0	2025-07-1...	2025-07-16 ...
4	1f37c497-2b0a-4c20-bf3f-8...	alice@example.com	Alice Wonderland	01710000001	P12345601	hash_pw1	true	2	2025-07-1...	2025-07-16 ...
5	d72a453b-e1cc-4bd8-84ee-...	bob@example.com	Bob Builder	01710000002	P12345602	hash_pw2	true	1	2025-07-1...	2025-07-16 ...
6	5e99a3c8-19e2-4a90-8049-...	charlie@example.com	Charlie Chaplin	01710000003	P12345603	hash_pw3	true	0	2025-07-1...	2025-07-16 ...
7	f441d68e-0a31-4b20-93d1-7...	diana@example.com	Diana Prince	01710000004	P12345604	hash_pw4	true	3	2025-07-1...	2025-07-16 ...
8	cc63af5e-fe6b-4873-b219-5...	ethan@example.com	Ethan Hunt	01710000005	P12345605	hash_pw5	true	5	2025-07-1...	2025-07-16 ...
9	b9dc1d92-d3aa-4db0-9880-...	fiona@example.com	Fiona Gallagher	01710000006	P12345606	hash_pw6	true	1	2025-07-1...	2025-07-16 ...

3.2.3 packages Table Snapshot

	id [PK] uuid	title character varying (255)	description text	price numeric (10,2)	destination_id uuid	featured_image character varying (500)	is_featured boolean	is_active boolean	created_at timestamp	updated_at timestamp w	created uuid
1	39f5d135-a1f3-434f-9ff...	Discover Sundar...	Explore the world's largest m...	4000.00	e9f85003-a946-4687-8811-8...	https://tywqqefmllgse...	false	true	2025-07-...	2025-07-...	3a2...
2	038a03ac-7383-4f96-bb...	Relaxing Beach ...	Enjoy the longest natural sea ...	1500.00	09e5b484-fdfb-466f-a0af-05...	https://tywqqefmllgse...	false	true	2025-07-...	2025-07-...	123...
3	b0b981cf-1859-4439-b6...	Mystic Sylhet Te...	Unwind in the serene landsca...	1500.00	7cce5e46-6227-455f-b4a3-f...	https://tywqqefmllgse...	false	true	2025-07-...	2025-07-...	123...
4	f3b2a1d1-9e0c-4b5d-9c...	Cox's Beach Bon...	3-day fun-filled tour to Cox's ...	12500.00	d1b5ea5a-12e9-4b52-8e3d-...	https://example.com/l...	true	true	2025-07-...	2025-07-...	a1a...
5	f3b2a1d1-9e0c-4b5d-9c...	Sajek Cloudy Es...	2-night stay in Sajek with sun...	10500.00	d1b5ea5a-12e9-4b52-8e3d-...	https://example.com/l...	true	true	2025-07-...	2025-07-...	a1a...
6	f3b2a1d1-9e0c-4b5d-9c...	Bandarban Trek...	4-day nature and hiking adve...	14500.00	d1b5ea5a-12e9-4b52-8e3d-...	https://example.com/l...	false	true	2025-07-...	2025-07-...	a1a...
7	f3b2a1d1-9e0c-4b5d-9c...	Saint Martin Get...	Island vibes and coral reef sn...	13500.00	d1b5ea5a-12e9-4b52-8e3d-...	https://example.com/l...	true	true	2025-07-...	2025-07-...	a1a...
8	f3b2a1d1-9e0c-4b5d-9c...	Jafong Nature ...	Green hills, river boating, and ...	9800.00	d1b5ea5a-12e9-4b52-8e3d-...	https://example.com/l...	true	true	2025-07-...	2025-07-...	a1a...

3.2.4 package-detail-schedule Table Snapshot

	package_id uuid	highlights text	itinerary text	inclusions text	exclusions text	terms_conditions text	duration_d integer	duration_h integer	max_gro integer	available_from timestamp without ti	available_until timestamp without tim	created_at timestamp without ti	updated timestamp
1	39f5d135-a1f3-4...	Boat ride, wildl...	Day 1: Arrival at K...	Accommodati...	Personal expen...	Bookings are non-r...	5	6	10	2025-07-15 00:00...	2025-07-24 00:00...	2025-07-15 16:5...	202...
2	038a03ac-7383-...	Sunbathing, be...	Day 1: Arrival and ...	Accommodati...	Personal expen...	Full refund availab...	5	5	5	2025-07-16 00:00...	2025-07-25 00:00...	2025-07-14 13:4...	202...
3	b0b981cf-1859-4...	Tea garden tou...	Day 1: Arrival in S...	Accommodati...	Lunch/dinner, p...	Advance booking r...	5	5	10	2025-07-16 00:00...	2025-07-30 00:00...	2025-07-15 18:1...	202...
4	f3b2a1d1-9e0c-4...	Beach BBQ, Se...	Day 1: Arrival in Da...	Hotel, BBQ, Tr...	Lunch	No refund after 7 d...	3	2	30	2025-07-16 01:38...	2025-10-16 01:38...	2025-07-16 01:3...	202...
5	f3b2a1d1-9e0c-4...	Sunrise trekkin...	Day 1: Travel in Da...	Stay, Breakfast	Guide fee	ID required	3	2	20	2025-07-16 01:38...	2025-09-16 01:38...	2025-07-16 01:3...	202...
6	f3b2a1d1-9e0c-4...	Waterfalls, hike	Day 1: Arrival in Da...	Guide, Food, T...	Shoes	No smoking	4	3	15	2025-07-16 01:38...	2026-01-16 01:38...	2025-07-16 01:3...	202...
7	f3b2a1d1-9e0c-4...	Boat ride, snor...	Day 1: Ferry in Da...	Resort, Meals	Alcohol	Wear safety gear	3	2	25	2025-07-16 01:38...	2025-11-16 01:38...	2025-07-16 01:3...	202...
8	f3b2a1d1-9e0c-4...	River boating, L...	Day 1: Travel in Da...	Hotel, Breakfast	Lunch	Family-friendly only	3	2	40	2025-07-16 01:38...	2025-12-16 01:38...	2025-07-16 01:3...	202...

3.2.5 package-images Table Snapshot

	id [PK] uuid	package_id uuid	image_url character varying (500)	alt_text character varying (255)	display_order integer	is_primary boolean	created_at timestamp without time zone
1	403bcf8b-d674-4cdf-81bc-f07524b32b9d	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	https://tywqqefmllgseuvdvoia.supabase...	[null]	0	true	2025-07-15 22:59:07.741379
2	4923bc3e-207f-43ea-9fce-0aa61d385029	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	https://tywqqefmllgseuvdvoia.supabase...	[null]	1	false	2025-07-15 22:59:07.753696
3	d456b708-e0ab-42e8-bd82-cb216759f0...	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	https://tywqqefmllgseuvdvoia.supabase...	[null]	2	false	2025-07-15 22:59:07.76166
4	ce8ecbecd-481b-401e-96ee-709130ea7c...	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	https://tywqqefmllgseuvdvoia.supabase...	[null]	0	true	2025-07-16 00:05:50.282862
5	0d8484d0-c948-4ce4-8c4c-ff40c474b667	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	https://tywqqefmllgseuvdvoia.supabase...	[null]	1	false	2025-07-16 00:05:50.291849
6	cb0287bf-558b-4d75-bca2-9e3f81a4ae38	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	https://tywqqefmllgseuvdvoia.supabase...	[null]	2	false	2025-07-16 00:05:50.30091
7	a407ea25-d66f-44de-9c39-a90b19abe2e1	b0b981cf-1859-4439-b6c1-2ecf7557307c	https://tywqqefmllgseuvdvoia.supabase...	[null]	0	true	2025-07-16 00:13:33.894445
8	059570fd-1e8a-4517-bb01-7f10261b30d9	b0b981cf-1859-4439-b6c1-2ecf7557307c	https://tywqqefmllgseuvdvoia.supabase...	[null]	1	false	2025-07-16 00:13:33.906443
9	2c28999f-c8e2-4a85-9b20-7d55a488f2be	b0b981cf-1859-4439-b6c1-2ecf7557307c	https://tywqqefmllgseuvdvoia.supabase...	[null]	2	false	2025-07-16 00:13:33.918448

3.2.6 bookings Table Snapshot

	id [PK] uuid	package_id uuid	user_id uuid	promo_code_id uuid	status character varying (20)	payment_status character varying	booking_date timestamp with time zone	cancellation_date timestamp with time zone	cancellation_text text	created_at timestamp without time zone	updated_at timestamp without time zone	totalAmount numeric (10,2)	paid_amount numeric (10, numeric	discount numeric
8	b7f8a9c0-2d...	f3b2a1d1-9e0...	cc63af5e-feb6...	[null]	confirmed	partial	2025-07-09...	[null]	[null]	2025-07-16 0...	2025-07-...	13500.00	6750.00	0.00
9	c1d2e3f4-5a...	f3b2a1d1-9e0...	b9dc1d92-d3aa...	9c32f1d0-e...	cancelled	refunded	2025-06-15...	2025-06-18...	Schedul...	2025-07-16 0...	2025-07-...	14500.00	0.00	0.00
10	d4e5f6a7-8b...	f3b2a1d1-9e0...	a95e7e2b-1e4a...	8b56a2f1-3...	confirmed	paid	2025-07-04...	[null]	[null]	2025-07-16 0...	2025-07-...	9800.00	9000.00	800.00
11	f37c43be-7d...	b0b981cf-185...	5a85f0a3-6cb0...	[null]	confirmed	paid	2025-07-14...	[null]	[null]	2025-07-14 2...	2025-07-...	3600.00	3600.00	0.00
12	0b44c318-98...	39f5d135-a1f...	5a85f0a3-6cb0...	[null]	pending	pending	2025-07-15...	[null]	[null]	2025-07-15 2...	2025-07-...	4000.00	0.00	0.00
13	08fb6eb3-dc...	038a03ac-738...	5a85f0a3-6cb0...	[null]	pending	pending	2025-07-15...	[null]	[null]	2025-07-15 2...	2025-07-...	1500.00	0.00	0.00
14	3e9383e6-3d...	39f5d135-a1f...	c361a0ec-603e...	[null]	pending	pending	2025-07-16...	[null]	[null]	2025-07-16 0...	2025-07-...	4000.00	0.00	0.00
15	a1b877f3-6e...	038a03ac-738...	c361a0ec-603e...	[null]	pending	pending	2025-07-16...	[null]	[null]	2025-07-16 0...	2025-07-...	1500.00	0.00	0.00
16	2247d110-b1...	b0b981cf-185...	c361a0ec-603e...	[null]	pending	pending	2025-07-16...	[null]	[null]	2025-07-16 0...	2025-07-...	1500.00	0.00	0.00

3.2.7 destinations Table Snapshot

	id [PK] uuid	name character varying (255)	country character varying (100)	city character varying (100)	description text	best_time_to_visit character varying (11)	featured_image text	is_active boolean	created_at timestamp without time zone	updated_at timestamp without time zone	created_by uuid
1	09e5b484-fd...	Cox's Bazar	Bangladesh	Chattogram	It's a goo...	December to Jan...	https://tywqqefmllgs...	true	2025-07-08 2...	2025-07-08 22:03:1...	123e456...
2	e9f85003-a9...	Sundarbans	Bangladesh	Khulna	The Sund...	October to March	https://tywqqefmllgs...	true	2025-07-15 2...	2025-07-15 22:49:2...	3a24275f...
3	7cce5e46-62...	Sylhet	Bangladesh	Sylhet	Sylhet, lo...	October to March	https://tywqqefmllgs...	true	2025-07-16 0...	2025-07-16 00:09:1...	3a24275f...
4	d1b5ea5a-12...	Cox's Bazar Beach	Bangladesh	Cox's Bazar	Longest ...	November - Febr...	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...
5	d1b5ea5a-12...	Mangrove Forest	Bangladesh	Khulna	Largest ...	October - March	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...
6	d1b5ea5a-12...	Sajek Valley	Bangladesh	Rangamati	Hill area ...	November - April	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...
7	d1b5ea5a-12...	Bandarban	Bangladesh	Bandarban	Trekking ...	November - Febr...	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...
8	d1b5ea5a-12...	Saint Martin's	Bangladesh	Teknaf	Small isl...	November - March	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...
9	d1b5ea5a-12...	Nilgiri	Bangladesh	Bandarban	Scenic m...	October - March	https://example.com...	true	2025-07-16 0...	2025-07-16 01:33:2...	a1a1a1a1...

3.2.8 promo-codes Table Snapshot

	id [PK] uuid	code character varying (2)	description text	discount_type character varying (2)	discount_value double precision	minimum_amount double precision	maximum_discount double precision	start_date date	expiry_date date	usage_limit integer	used_count integer	is_active boolean	created_at timestamp without time zone	updated_at timestamp without time zone	created_by uuid
1	0115e54...	SAVE20	Wellcome	percentage	20	1500	2000	2025-07-08	2025-08-07	2	0	true	2025-07-08 18:...	2025-...	123...
2	e010b1c1...	SUMMER25	25% off summer	percentage	25	1000	500	2025-07-01	2025-08-31	100	12	true	2025-07-16 01:...	2025-...	a1a...
3	e010b1c1...	WELCOME100	100 off for new	fixed	100	0	[null]	2025-01-01	2025-12-31	500	80	true	2025-07-16 01:...	2025-...	a1a...
4	e010b1c1...	EARLYBIRD	Flat 10% for early	percentage	10	2000	300	2025-06-01	2025-09-01	300	22	true	2025-07-16 01:...	2025-...	a1a...
5	e010b1c1...	MONSOON50	50 off on monso	fixed	50	500	100	2025-07-10	2025-09-10	200	30	true	2025-07-16 01:...	2025-...	a1a...
6	e010b1c1...	REFER20	20% off via refer	percentage	20	1000	300	2025-04-01	2025-12-31	1000	200	true	2025-07-16 01:...	2025-...	a1a...
7	e010b1c1...	BLACKFRIDAY	Black Friday deal	fixed	300	3000	[null]	2025-11-25	2025-11-30	100	50	true	2025-07-16 01:...	2025-...	a1a...
8	e010b1c1...	SPRING15	Spring trip 15% off	percentage	15	1500	400	2025-03-01	2025-05-31	250	99	true	2025-07-16 01:...	2025-...	a1a...
9	e010b1c1...	FAMILYFUN	1200 off for grou	fixed	200	3000	200	2025-05-01	2025-12-01	300	50	true	2025-07-16 01:...	2025-...	a1a...

3.2.9 blogs Table Snapshot

	id [PK] uuid	title character varying (255)	author_id uuid	content text	excerpt text	status character varying (20)	published_at timestamp without time zone	category character varying (20)	cover_image text	is_featured boolean	created_at timestamp without time zone	updated_at timestamp without time zone
5	b101ec1a-1001-4bc...	A Weekend in ...	144be6a9-8...	Experience Sajek fr...	Tips for travelin...	published	2025-07-02 00:0...	Nature	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
6	b101ec1a-1001-4bc...	Saint Martin's I...	c361a0ec-6...	How to reach, what ...	Ultimate travel g...	published	2025-06-28 00:0...	Island	https://example.c...	true	2025-07-16 01:40:5...	2025-07-...
7	b101ec1a-1001-4bc...	Best Adventur...	1f37c497-2...	Nilgiri, Nafakhum a...	Hidden trekking ...	published	2025-07-03 00:0...	Adventure	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
8	b101ec1a-1001-4bc...	Budget Travel ...	d72a453b-e...	How to save while tr...	Student discount...	published	2025-06-30 00:0...	Tips	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
9	b101ec1a-1001-4bc...	Monsoon Dest...	5e99a3c8-1...	Best rainy season s...	When to travel i...	published	2025-07-05 00:0...	Seasonal	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
10	b101ec1a-1001-4bc...	Travel Essent...	f441d68e-0...	What to pack before...	Don't forget the...	draft	[null]	Tips	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
11	b101ec1a-1001-4bc...	Exploring Jati...	cc63af5e-fe...	Tea gardens, rivers...	What to do in Ja...	published	2025-07-06 00:0...	Culture	https://example.c...	true	2025-07-16 01:40:5...	2025-07-...
12	b101ec1a-1001-4bc...	7 Mistakes Fir...	b9dc1d92-d...	Avoid these commo...	Pack smart, pla...	published	2025-07-07 00:0...	Advice	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...
13	b101ec1a-1001-4bc...	How to Plan a ...	a95e7e2b-1...	Organizing group to...	Roles, budgetin...	published	2025-07-08 00:0...	Group Travel	https://example.c...	false	2025-07-16 01:40:5...	2025-07-...

4 Query Examples

4.1 Basic Queries with Joins

4.1.1 Cross Product Example

```

1 -- Query: Find all possible pairs of users and packages available in
  the system.
2 SELECT u.uid, u.full_name, p.id AS package_id, p.title
3 FROM users u
4 CROSS JOIN packages p;
```

Listing 10: Cross Product - All User-Packages Combinations

Relational Algebra:

$$\pi_{uid,full_name,id,title} (users \times packages)$$

Query Output:

	uid uuid	full_name character varying (255)	package_id uuid	title character varying (255)
5	5a85f0a3-6cb0-4578-b6ec-70a3813609...	Mehedi Hasan	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape
6	5a85f0a3-6cb0-4578-b6ec-70a3813609...	Mehedi Hasan	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111003	Bandarban Trekking
7	5a85f0a3-6cb0-4578-b6ec-70a3813609...	Mehedi Hasan	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway
8	5a85f0a3-6cb0-4578-b6ec-70a3813609...	Mehedi Hasan	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jafiong Nature Tour
9	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure
10	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar
11	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat
12	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	Cox's Beach Bonanza
13	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape
14	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111003	Bandarban Trekking
15	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway
16	144be6a9-8794-46d6-9a9c-86ed44fb0b...	New	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jafiong Nature Tour
17	c361a0ec-603e-482a-b52e-795d12caddf0	Abu Bakar Siddique	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure
18	c361a0ec-603e-482a-b52e-795d12caddf0	Abu Bakar Siddique	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar

4.1.2 Left Outer Join Example

```

1 -- Query: Retrieve all packages along with their booking information (
  if any).
2 -- Show even those packages that have never been booked.
3 SELECT p.id AS package_id, p.title, b.id AS booking_id, b.user_id, b.
  status
4 FROM packages p
5 LEFT OUTER JOIN bookings b ON p.id = b.package_id;

```

Listing 11: Left Outer Join - All Packages With or Without Bookings

Relational Algebra:

$$\pi_{p.id, p.title, b.id, b.user_id, b.status} (packages \bowtie_{packages.id=bookings.package_id} bookings)$$

Query Output:

	package_id uuid	title character varying (255)	booking_id uuid	user_id uuid	status character varying (20)
1	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Cox's Beach Bonanza	a9f2a1d1-3e9c-4f5b-b9c1-6d7e8f111001	5a85f0a3-6cb0-4578-b6ec-70a381360965	confirmed
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Sajek Cloudy Escape	b0c8b8d9-5e6f-4e8c-b0d3-1a2b3c4d5002	144be6a9-8794-46d6-9a9c-86ed44fb0b6f	pending
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Bandarban Trekking	c7a2f0d1-1b2c-4d5e-9f8a-0b1c2d3e4003	c361a0ec-603e-482a-b52e-795d12caddf0	cancelled
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Saint Martin Getaway	d5b7f2a9-7f5e-4a9b-8c7d-1e2f3a4b6004	1f37c497-2b0a-4c20-b5f3-88a3ad122001	confirmed
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Jafiong Nature Tour	e0c9a3d2-8d7b-4a8f-bcd1-2e3f4a5b7005	d72a453b-e1cc-4bd8-84ea-1b66f0202002	confirmed
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Cox's Beach Bonanza	f2d3b1a0-9c4e-4a8d-b9d2-3c4e5f6a8006	5e99a3c8-19e2-4a90-8049-344fcb32003	confirmed
7	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Sajek Cloudy Escape	a5e6c7d8-1f2a-4b3c-9d4e-5f6a7b8c9007	f441d68e-0a31-4b20-93d1-7d1c2e440004	pending
8	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Saint Martin Getaway	b7f8a9c0-2d3e-4f5a-8b9c-0d1e2f3a4008	cc63af5e-feb6-4873-b219-507ac8110005	confirmed
9	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Bandarban Trekking	c1d2e3f4-5a6b-7c8d-9e0f-1a2b3c4d5009	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	cancelled
10	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Jafiong Nature Tour	d4e5f6a7-8b9c-0d1e-2f3a-4b5c6d7e8010	a95e7e2b-1e4a-41e9-8e11-7a598ea600...	confirmed
11	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	f37c43be-7d5e-4153-9b83-9be8d6133ea2	5a85f0a3-6cb0-4578-b6ec-70a381360965	confirmed
12	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	0b44c318-9822-4b8a-8651-f284b0403676	5a85f0a3-6cb0-4578-b6ec-70a381360965	pending
13	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	08fb6eb3-dc56-4bdd-9305-078ced980f47	5a85f0a3-6cb0-4578-b6ec-70a381360965	pending
14	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	3e9383e6-3d1b-4449-96b5-72664e683d...	c361a0ec-603e-482a-b52e-795d12caddf0	pending

4.1.3 Join with ON Example

```

1 -- Query: Retrieve booking info with package title and price
2 SELECT b.id AS booking_id, b.user_id, p.title, p.price
3 FROM bookings b
4 JOIN packages p ON b.package_id = p.id;

```

Listing 12: Join with ON - Join Bookings and Packages by Package ID

Relational Algebra:

$$\pi_{b.id, b.user_id, p.title, p.price}(\text{bookings} \bowtie_{\text{bookings.package_id}=\text{packages.id}} \text{packages})$$
Query Output:

	booking_id uuid	user_id uuid	title character varying (255)	price numeric (10,2)
1	a9f2a1d1-3e9c-4f5b-b9c1-6d7e8f111001	5a85f0a3-6cb0-4578-b6ec-70a381360965	Cox's Beach Bonanza	12500.00
2	b0c8b8d9-5e6f-4e8c-b0d3-1a2b3c4d5002	144be6a9-8794-46d6-9a9c-86ed44fb0b6f	Sajek Cloudy Escape	10500.00
3	c7a2f0d1-1b2c-4d5e-9f8a-0b1c2d3e4003	c361a0ec-603e-482a-b52e-795d12caddf0	Bandarban Trekking	14500.00
4	d5b7f2a9-7f5e-4a9b-8c7d-1e2f3a4b6004	1f37c497-2b0a-4c20-b5f3-88a3ad122001	Saint Martin Getaway	13500.00
5	e0c9a3d2-8d7b-4a8f-bcd1-2e3f4a5b7005	d72a453b-e1cc-4bd8-84ea-1b66f0202002	Jaflong Nature Tour	9800.00
6	f2d3b1a0-9c4e-4a8d-b9d2-3c4e5f6a8006	5e99a3c8-19e2-4a90-8049-344fcfb32003	Cox's Beach Bonanza	12500.00
7	a5e6c7d8-1f2a-4b3c-9d4e-5f6a7b8c9007	f441d68e-0a31-4b20-93d1-7d1c2e440004	Sajek Cloudy Escape	10500.00
8	b7f8a9c0-2d3e-4f5a-8b9c-0d1e2f3a4008	cc63af5e-feb6-4873-b219-507ac8110005	Saint Martin Getaway	13500.00
9	c1d2e3f4-5a6b-7c8d-9e0f-1a2b3c4d5009	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	Bandarban Trekking	14500.00
10	d4e5f6a7-8b9c-0d1e-2f3a-4b5c6d7e8010	a95e7e2b-1e4a-41e9-86e1-7a598ea600...	Jaflong Nature Tour	9800.00
11	f37c43be-7d5e-4153-9b83-9be8d6133ea2	5a85f0a3-6cb0-4578-b6ec-70a381360965	Mystic Sylhet Tea Garden Retreat	1500.00
12	0b44c318-9822-4b8a-8651-f284b0403676	5a85f0a3-6cb0-4578-b6ec-70a381360965	Discover Sundarbans Adventure	4000.00
13	08f6eb3-dc56-4bdd-9305-078ced980f47	5a85f0a3-6cb0-4578-b6ec-70a381360965	Relaxing Beach Holiday at Cox's Bazar	1500.00
14	3e9383e6-3d1b-4449-96b5-72664e683d...	c361a0ec-603e-482a-b52e-795d12caddf0	Discover Sundarbans Adventure	4000.00

4.2 Nested Subqueries

4.2.1 Subquery with ANY Example

```

1 -- Query: Find packages where the price is less than any paid amount in
   bookings
2 SELECT p.id, p.title, p.price
3 FROM packages p
4 WHERE p.price < ANY (
5     SELECT b.paid_amount
6     FROM bookings b
7 );

```

Listing 13: Find packages priced less than any booking's paid amount

Relational Algebra:

Expressing ANY in relational algebra is not straightforward, but the query conceptually means:

$$\{p \in \text{packages} \mid \exists b \in \text{bookings} \text{ such that } p.\text{price} < b.\text{paid_amount}\}$$

Or as a selection:

$$\sigma_{\text{price} < \text{paid_amount}}(\text{packages} \times \text{bookings})$$

followed by a projection on package attributes with the condition that **at least one booking's paid amount is greater than package price**.

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	4000.00
2	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	1500.00
3	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape	10500.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jaflong Nature Tour	9800.00

4.2.2 Subquery with SOME Example

```

1 -- Query: Find packages priced less than some packages priced over 1000
2 SELECT p1.id, p1.title, p1.price
3 FROM packages p1
4 WHERE p1.price < SOME (
5     SELECT p2.price
6     FROM packages p2
7     WHERE p2.price > 1000
8 );

```

Listing 14: Find packages cheaper than some packages with high price

Relational Algebra:

$$\pi_{id, title, price} \left(\sigma_{p1.price < p2.price \wedge p2.price > 1000} (packages_{p1} \times packages_{p2}) \right)$$

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	4000.00
2	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	1500.00
3	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	Cox's Beach Bonanza	12500.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape	10500.00
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway	13500.00
7	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jaflong Nature Tour	9800.00

4.2.3 Subquery with ALL Example

```

1 -- Query: Find packages priced less than or equal to all packages
  priced over 500
2 SELECT p1.id, p1.title, p1.price
3 FROM packages p1
4 WHERE p1.price <= ALL (
5     SELECT p2.price
6     FROM packages p2
7     WHERE p2.price > 500
8 );

```

Listing 15: Find packages priced less than or equal to all packages priced over 500

Relational Algebra:

$$\pi_{id, title, price} \left(\sigma_{\forall p2 (p2.price > 500 \implies p1.price \leq p2.price)} (packages_{p1}) \right)$$

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	1500.00
2	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00

4.2.4 Subquery with EXISTS Example

```

1 -- Query: Find packages for which there exists at least one booking
2 SELECT p.id, p.title, p.price
3 FROM packages p
4 WHERE EXISTS (
5     SELECT 1
6     FROM bookings b
7     WHERE b.package_id = p.id
8 );

```

Listing 16: Find packages that have at least one booking

Relational Algebra:

$$\pi_{id, title, price} \left(\sigma_{\exists b \in bookings, b.package_id = p.id} (packages) \right)$$

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	4000.00
2	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	1500.00
3	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	Cox's Beach Bonanza	12500.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape	10500.00
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111003	Bandarban Trekking	14500.00
7	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway	13500.00
8	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jaflong Nature Tour	9800.00

4.3 Subqueries in Different Clauses**4.3.1 Subquery in FROM Clause Example**

```

1 -- Query: Find the average paid amount per package using a subquery in
  FROM clause
2 SELECT sub.package_id, sub.avg_paid_amount
3 FROM (
4     SELECT b.package_id, AVG(b.paid_amount) AS avg_paid_amount
5     FROM bookings b

```



```

6      GROUP BY b.package_id
7    ) sub
8 JOIN packages p ON sub.package_id = p.id;

```

Listing 17: Calculate average booking amount per package

Relational Algebra:

1. Compute the aggregation (average) grouped by package-id:

$$\gamma_{\text{package_id}; \text{AVG}(\text{paid_amount}) \rightarrow \text{avg_paid_amount}}(\text{bookings})$$

2. Join the result with packages on package-id = id:

$$\pi_{\text{sub.package_id}, \text{sub.avg_paid_amount}}(\gamma_{\text{package_id}; \text{AVG}(\text{paid_amount})}(\text{bookings})) \bowtie_{\text{sub.package_id}=\text{packages.id}} \text{packages}$$

Query Output:

	package_id uuid	avg_paid_amount numeric
1	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	0.00000000000000000000
2	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	0.00000000000000000000
3	b0b981cf-1859-4439-b6c1-2ecf7557307c	1800.0000000000000000
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	6000.0000000000000000
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	0.00000000000000000000
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111003	0.00000000000000000000
7	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	6875.0000000000000000
8	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	9400.0000000000000000

4.3.2 Subquery in WHERE Clause Example

```

1 -- Query: Find packages that have at least one booking with paid_amount
  > 1000
2 SELECT p.id, p.title, p.price
3 FROM packages p
4 WHERE p.id IN (
5     SELECT b.package_id
6     FROM bookings b
7     WHERE b.paid_amount > 1000
8 );

```

Listing 18: Find packages that have bookings with paid amount greater than 1000

Relational Algebra:

1. Select bookings with paid amount ≥ 1000 :

$$\sigma_{\text{paid_amount} \geq 1000}(\text{bookings})$$

2. Project package IDs from these bookings:

$$\pi_{\text{package_id}}(\sigma_{\text{paid_amount} > 1000}(\text{bookings}))$$

3. Select packages whose id is in the set of package IDs:

$$\sigma_{\text{id} \in \pi_{\text{package_id}}(\sigma_{\text{paid_amount} > 1000}(\text{bookings}))}(\text{packages})$$

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	Cox's Beach Bonanza	12500.00
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway	13500.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jaflong Nature Tour	9800.00

4.3.3 Subquery in SELECT Clause Example

```

1 -- Query: List packages with the number of bookings for each
2 SELECT p.id, p.title,
3       (SELECT COUNT(*)
4        FROM bookings b
5        WHERE b.package_id = p.id) AS booking_count
6 FROM packages p;
```

Listing 19: Show packages with the count of bookings

Relational Algebra:

1. Count bookings per package:

$$\gamma_{\text{package_id}; \text{COUNT}(\ast) \rightarrow \text{booking_count}}(\text{bookings})$$

2. Left join packages with the aggregated counts (to include packages with zero bookings):

$$\text{packages} \bowtie_{\text{packages.id}=\text{bookings.package_id}} \gamma_{\text{package_id}; \text{COUNT}(\ast)}(\text{bookings})$$

3. Project package id, title, and booking count:

$$\pi_{\text{id}, \text{title}, \text{booking_count}}(\dots)$$

Query Output:

	id [PK] uuid	title character varying (255)	booking_count bigint
1	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	2
2	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	2
3	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	2
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	Cox's Beach Bonanza	2
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111002	Sajek Cloudy Escape	2
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111003	Bandarban Trekking	2
7	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	Saint Martin Getaway	2
8	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	Jaflong Nature Tour	2

4.4 Advanced Clauses

4.4.1 ORDER BY Clause Example

```

1 -- Query: Show all active packages sorted by highest price first
2 SELECT id, title, price
3 FROM packages
4 WHERE is_active = true
5 ORDER BY price DESC;

```

Listing 20: List packages ordered by price descending

Relational Algebra:

1. Select only active packages:

$$\sigma_{\text{is_active}=\text{true}}(\text{packages})$$

2. Project relevant attributes:

$$\pi_{\text{id}, \text{title}, \text{price}}(\dots)$$

3. Apply sorting by price in descending order (denoted as τ):

$$\tau_{\text{price desc}}(\pi_{\text{id}, \text{title}, \text{price}}(\sigma_{\text{is_active}=\text{true}}(\text{packages})))$$

Query Output:

	id [PK] uuid	title character varying (255)	price numeric (10,2)
1	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Bandarban Trekking	14500.00
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Saint Martin Getaway	13500.00
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Cox's Beach Bonanza	12500.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Sajek Cloudy Escape	10500.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Jaflong Nature Tour	9800.00
6	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	4000.00
7	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	1500.00
8	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	1500.00

4.4.2 GROUP BY Clause Example

```

1 -- Query: Find total revenue (sum of paid_amount) grouped by package
2 SELECT package_id, SUM(paid_amount) AS total_revenue
3 FROM bookings
4 GROUP BY package_id;

```

Listing 21: Total revenue generated per package

Relational Algebra:

1. Group by package-id and aggregate SUM(paid-amount):

$$\gamma_{\text{package_id}; \text{SUM}(\text{paid_amount}) \rightarrow \text{total_revenue}}(\text{bookings})$$

2. Project the result (optional if already just these attributes):

$$\pi_{\text{package_id}, \text{total_revenue}}(\dots)$$

Query Output:

	package_id uuid	total_revenue numeric
1	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	0.00
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	18800.00
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	13750.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	12000.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	0.00
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	0.00
7	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	0.00
8	b0b981cf-1859-4439-b6c1-2ecf7557307c	3600.00

4.4.3 HAVING Clause Example

```

1 -- Query: Show packages that generated more than 10,000 in total
  revenue
2 SELECT package_id, SUM(paid_amount) AS total_revenue
3 FROM bookings
4 GROUP BY package_id
5 HAVING SUM(paid_amount) > 10000;

```

Listing 22: Packages with total revenue over 10

Relational Algebra:

1. Group by package-id and compute SUM(paid-amount):

$$G := \gamma_{\text{package_id}; \text{SUM}(\text{paid_amount}) \rightarrow \text{total_revenue}}(\text{bookings})$$

2. Apply selection on the grouped result:

$$\sigma_{\text{total_revenue} > 10000}(G)$$

3. Project final result:

$$\pi_{\text{package_id}, \text{total_revenue}}(\dots)$$

Query Output:

	package_id uuid	total_revenue numeric
1	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111005	18800.00
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111004	13750.00
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a111001	12000.00

4.4.4 WITH Clause (CTE) Example

```

1 -- Query: Use a CTE to calculate total revenue per package and join
  with packages
2 WITH revenue_per_package AS (
3     SELECT package_id, SUM(paid_amount) AS total_revenue
4     FROM bookings
5     GROUP BY package_id
6 )
7 SELECT p.id, p.title, COALESCE(r.total_revenue, 0) AS total_revenue
8 FROM packages p
9 LEFT JOIN revenue_per_package r ON p.id = r.package_id;

```

Listing 23: List packages with their total revenue using CTE

Relational Algebra:

1. Compute revenue per package as a temporary relation R :

$$R := \gamma_{\text{package_id}; \text{SUM}(\text{paid_amount}) \rightarrow \text{total_revenue}}(\text{bookings})$$

2. Left outer join with packages to include all packages:

$$J := \text{packages} \bowtie_{\text{packages.id}=R.\text{package_id}} R$$

3. Project the desired attributes:

$$\pi_{\text{id}, \text{title}, \text{total_revenue}}(J)$$

Query Output:

	id [PK] uuid	title character varying (255)	total_revenue numeric
1	038a03ac-7383-4f96-bbe9-f5c7b2bc356e	Relaxing Beach Holiday at Cox's Bazar	0.00
2	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Jaflong Nature Tour	18800.00
3	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Saint Martin Getaway	13750.00
4	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Cox's Beach Bonanza	12000.00
5	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Sajek Cloudy Escape	0.00
6	f3b2a1d1-9e0c-4b5d-9c51-1c2e3a1110...	Bandarban Trekking	0.00
7	39f5d135-a1f3-434f-9ff8-ce34e4fd7e26	Discover Sundarbans Adventure	0.00
8	b0b981cf-1859-4439-b6c1-2ecf7557307c	Mystic Sylhet Tea Garden Retreat	3600.00

4.5 String and Set Operations

4.5.1 String Operations Example 4

```

1 -- Query: Select users whose full name contains 'Bakar' as a whole word
2 SELECT full_name
3 FROM users
4 WHERE full_name ILIKE '% Bakar %';

```

Listing 24: Find users with the word 'Bakar' in their name

Relational Algebra (Conceptual):

1. Use selection with a string match condition:

$$\sigma_{\text{full_name ILIKE '% Bakar \%'}}(\text{users})$$

2. Project only the full name:

$$\pi_{\text{full_name}}(\dots)$$

Query Output:

	full_name character varying (255)
1	Abu Bakar Siddique

4.6 Update and Delete Operations

4.6.1 Simple UPDATE Example

```

1 -- Query: Update full name of a specific user
2 UPDATE users
3 SET full_name = 'Farhan Ahmed'
4 WHERE email = 'alice@example.com';

```

Listing 25: Update a user's full name

Before Update:

	uid [PK] uuid	email character varying (255)	full_name character varying (255)	phone character varying (20)	passport character varying (255)
1	5a83f0a3-6cb0-4578-b6ec-70a3813609...	hasanmehedi26696@gmail.com	Mehedi Hasan	+8801319926696	12345678
2	144be6a9-8794-46d5-9a9c-86ed44f0b0...	jfwjrw@gmail.com	New	+8801319926696	2375983
3	c361a0ec-603e-482a-b52e-795d12cad...	bojackabs@gmail.com	Abu Bakar Siddique	+8801319926657	5789273982
4	1f37c497-2b0a-4c20-b5f3-88a3ad1220...	alice@example.com	Alice Wonderland	01710000001	P12345601
5	d72a453b-e1cc-4bd8-84ea-1b66f02020...	bob@example.com	Bob Builder	01710000002	P12345602
6	5e99a3c8-19e2-4a90-8049-344fcfb32003	charlie@example.com	Charlie Chaplin	01710000003	P12345603
7	f441d68e-0a31-4b20-93d1-7d1c2e4400...	diana@example.com	Diana Prince	01710000004	P12345604
8	cc63af5e-f6b6-4873-b219-507ac8110005	ethan@example.com	Ethan Hunt	01710000005	P12345605
9	b96c1d92-d3aa-4db0-9880-91d7ed550...	fiona@example.com	Fiona Gallagher	01710000006	P12345606

After Update:

	uid [PK] uuid	email character varying (255)	full_name character varying (255)	phone character varying (20)	passport character varying (255)
5	5e99a3c8-19e2-4a90-8049-3441cb32003	charlie@example.com	Charlie Chaplin	01710000003	P12345603
6	f441d68e-0a31-4b20-93d1-7d1c2e4400...	diana@example.com	Diana Prince	01710000004	P12345604
7	cc63af5e-feb6-4873-b219-507ac8110005	ethan@example.com	Ethan Hunt	01710000005	P12345605
8	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	fiona@example.com	Fiona Gallagher	01710000006	P12345606
9	a95e7e2b-1e4a-41e9-86e1-7a598ea600...	george@example.com	George Orwell	01710000007	P12345607
10	7777cbbc-464e-46de-8826-0b4c10990008	hannah@example.com	Hannah Montana	01710000008	P12345608
11	89b01e4b-4690-4a4a-ae6e-87f117d00009	ivan@example.com	Ivan Drago	01710000009	P12345609
12	3d65de21-cfc6-4f78-b83f-57eb2b2b0010	julia@example.com	Julia Roberts	01710000010	P12345610
13	1f37c497-2b0a-4c20-b5f3-88a3ad122001	alice@example.com	Farhan Ahmed	01710000001	P12345601

4.6.2 DELETE Query Example

```

1 -- Query: Delete a user from the system by email
2 DELETE FROM users
3 WHERE email = 'alice@example.com';

```

Listing 26: Delete a user by email

Before Deletion:

	uid [PK] uuid	email character varying (255)	full_name character varying (255)	phone character varying (20)	passport character varying (255)
5	5e99a3c8-19e2-4a90-8049-3441cb32003	charlie@example.com	Charlie Chaplin	01710000003	P12345603
6	f441d68e-0a31-4b20-93d1-7d1c2e4400...	diana@example.com	Diana Prince	01710000004	P12345604
7	cc63af5e-feb6-4873-b219-507ac8110005	ethan@example.com	Ethan Hunt	01710000005	P12345605
8	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	fiona@example.com	Fiona Gallagher	01710000006	P12345606
9	a95e7e2b-1e4a-41e9-86e1-7a598ea600...	george@example.com	George Orwell	01710000007	P12345607
10	7777cbbc-464e-46de-8826-0b4c10990008	hannah@example.com	Hannah Montana	01710000008	P12345608
11	89b01e4b-4690-4a4a-ae6e-87f117d00009	ivan@example.com	Ivan Drago	01710000009	P12345609
12	3d65de21-cfc6-4f78-b83f-57eb2b2b0010	julia@example.com	Julia Roberts	01710000010	P12345610
13	1f37c497-2b0a-4c20-b5f3-88a3ad122001	alice@example.com	Farhan Ahmed	01710000001	P12345601

After Deletion:

	uid [PK] uuid	email character varying (255)	full_name character varying (255)	phone character varying (20)	passport character varying (255)
4	072a453b-e1cc-4bd8-84ea-1b66f02020...	bob@example.com	Bob Builder	01710000002	P12345602
5	5e99a3c8-19e2-4a90-8049-3441cb32003	charlie@example.com	Charlie Chaplin	01710000003	P12345603
6	f441d68e-0a31-4b20-93d1-7d1c2e4400...	diana@example.com	Diana Prince	01710000004	P12345604
7	cc63af5e-feb6-4873-b219-507ac8110005	ethan@example.com	Ethan Hunt	01710000005	P12345605
8	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	fiona@example.com	Fiona Gallagher	01710000006	P12345606
9	a95e7e2b-1e4a-41e9-86e1-7a598ea600...	george@example.com	George Orwell	01710000007	P12345607
10	7777cbbc-464e-46de-8826-0b4c10990008	hannah@example.com	Hannah Montana	01710000008	P12345608
11	89b01e4b-4690-4a4a-ae6e-87f117d00009	ivan@example.com	Ivan Drago	01710000009	P12345609
12	3d65de21-cfc6-4f78-b83f-57eb2b2b0010	julia@example.com	Julia Roberts	01710000010	P12345610

4.7 Built-in Functions and Aggregates

4.7.1 Aggregate Functions Example

```

1 -- Query: Find total number of bookings and average paid amount per
  user
2 SELECT user_id,
3         COUNT(*) AS total_bookings,
4         AVG(paid_amount) AS avg_paid_amount
5 FROM bookings
6 GROUP BY user_id;

```

Listing 27: Count and average bookings per user

Relational Algebra:

1. Group bookings by user-id and compute aggregates:

$$\gamma_{\text{user_id}; \text{COUNT}(\ast) \rightarrow \text{total_bookings}, \text{AVG}(\text{paid_amount}) \rightarrow \text{avg_paid_amount}}(\text{bookings})$$

2. Project relevant attributes (if needed):

$$\pi_{\text{user_id}, \text{total_bookings}, \text{avg_paid_amount}}(\dots)$$

Query Output:

	user_id uuid	total_bookings bigint	avg_paid_amount numeric
2	b9dc1d92-d3aa-4db0-9880-91d7ed5500...	1	0.00000000000000000000
3	cc63af5e-feb6-4873-b219-507ac8110005	1	6750.0000000000000000
4	5a85f0a3-6cb0-4578-b6ec-70a381360965	4	3900.0000000000000000
5	a95e7e2b-1e4a-41e9-86e1-7a598ea60007	1	9000.0000000000000000
6	1f37c497-2b0a-4c20-b5f3-88a3ad122001	1	7000.0000000000000000
7	c361a0ec-603e-482a-b52e-795d12caddf0	4	0.00000000000000000000
8	d72a453b-e1cc-4bd8-84ea-1b66f0202002	1	9800.0000000000000000
9	5e99a3c8-19e2-4a90-8049-344fcfb32003	1	0.00000000000000000000
10	f441d68e-0a31-4b20-93d1-7d1c2e440004	1	0.00000000000000000000

4.7.2 Date and Time Functions Example

```

1 -- Query: Count bookings grouped by booking month for the year 2025
2 SELECT
3     DATE_TRUNC('month', booking_date) AS booking_month,
4     COUNT(*) AS bookings_count
5 FROM bookings
6 WHERE booking_date >= '2025-01-01' AND booking_date < '2026-01-01'
7 GROUP BY booking_month
8 ORDER BY booking_month;

```

Listing 28: Count bookings per month in 2025

Relational Algebra:

1. Select bookings in 2025:

$$\sigma_{\text{booking_date} \geq '2025-01-01' \wedge \text{booking_date} < '2026-01-01'}(\text{bookings})$$

2. Project booking month (via truncation) and count bookings per month (grouping):

$$\gamma_{\text{booking_month}; \text{COUNT}(\ast) \rightarrow \text{bookings_count}}(\dots)$$

3. Order results by month (sorting operator τ):

$$\tau_{\text{booking_month}}(\dots)$$

Query Output:

	booking_month timestamp without time zone	bookings_count bigint
1	2025-06-01 00:00:00	2
2	2025-07-01 00:00:00	14

5 Views Implementation

5.1 View Creation

5.1.1 CREATE VIEW with Joins and Aggregates Example

```

1 CREATE VIEW package_summary AS
2 SELECT
3     p.id,
4     p.title,
5     p.price,
6     p.is_featured,
7     d.name AS destination_name,
8     d.country,
9     d.city,
10    COUNT(b.id) AS total_bookings,
11    COALESCE(SUM(b.total_amount), 0) AS total_revenue,
12    COALESCE(AVG(b.total_amount), 0) AS avg_booking_value,
13    p.created_at
14 FROM packages p
15 JOIN destinations d ON p.destination_id = d.id
16 LEFT JOIN bookings b ON p.id = b.package_id AND b.status = 'confirmed'
17 WHERE p.is_active = true
18 GROUP BY
19     p.id, p.title, p.price, p.is_featured,
20     d.name, d.country, d.city,
21     p.created_at;

```

Listing 29: Create a view summarizing package booking data with destination info

Relational Algebra:

1. Join packages and destinations:

$$J_1 := \text{packages} \bowtie_{\text{packages.destination_id}=\text{destinations.id}} \text{destinations}$$

2. Left join with bookings filtered by confirmed status:

$$J_2 := J_1 \bowtie_{\text{packages.id}=\text{bookings.package_id} \wedge \text{bookings.status}='confirmed'} \text{bookings}$$

3. Select only active packages:

$$\sigma_{\text{is_active}=\text{true}}(J_2)$$

4. Group by relevant attributes and aggregate booking counts and sums:

$$\gamma_{\substack{p.id, p.title, p.price, p.is_featured, \\ d.name, d.country, d.city, p.created_at}} \left\{ \begin{array}{l} \text{COUNT}(b.id) \rightarrow \text{total_bookings}, \\ \text{SUM}(b.total_amount) \rightarrow \text{total_revenue}, \\ \text{AVG}(b.total_amount) \rightarrow \text{avg_booking_value} \end{array} \right. \quad (\dots)$$

Query Output:

	id uuid	title character varying (255)	price numeric (10,2)	is_featured boolean	destination_name character varying (255)	country character varying (100)	city character varying (100)	total_boc bigint	total_revenue numeric	avg_booking numeric	created_at timestamp without time zone
1	038a03ac-7383-...	Relaxing Beach Holiday ...	1500.00	false	Cox's Bazar	Bangladesh	Chattogram	0	0	0	2025-07-08 23:20:34.292412
2	39f5d135-a1f3-4...	Discover Sundarbans A...	4000.00	false	Sundarbans	Bangladesh	Khulna	0	0	0	2025-07-15 22:59:07.700933
3	b0b981cf-1859-4...	Mystic Sylhet Tea Garde...	1500.00	false	Sylhet	Bangladesh	Sylhet	1	3600.00	3600.000000	2025-07-08 22:08:47.930273
4	f3b2a1d1-9e0c-4...	Cox's Beach Bonanza	12500.00	true	Cox's Bazar Beach	Bangladesh	Cox's Bazar	2	25000.00	12500.000000	2025-07-16 01:37:25.036793
5	f3b2a1d1-9e0c-4...	Sajek Cloudy Escape	10500.00	true	Sajek Valley	Bangladesh	Rangamati	0	0	0	2025-07-16 01:37:25.036793
6	f3b2a1d1-9e0c-4...	Bandarban Trekking	14500.00	false	Bandarban	Bangladesh	Bandarban	0	0	0	2025-07-16 01:37:25.036793
7	f3b2a1d1-9e0c-4...	Saint Martin Getaway	13500.00	true	Saint Martin's	Bangladesh	Teknaf	2	27000.00	13500.000000	2025-07-16 01:37:25.036793
8	f3b2a1d1-9e0c-4...	Jafong Nature Tour	9800.00	true	Jafong	Bangladesh	Sylhet	2	19600.00	9800.000000	2025-07-16 01:37:25.036793

5.1.2 CREATE VIEW for User Activity Summary

```

1 CREATE VIEW user_activity_summary AS
2 SELECT
3     u.uid,
4     u.full_name,
5     u.email,
6     u.is_active,
7     COUNT(b.id) AS total_bookings,
8     COALESCE(SUM(b.total_amount), 0) AS total_spent,
9     COALESCE(AVG(b.total_amount), 0) AS avg_spending,
10    MAX(b.booking_date) AS last_booking_date,
11    u.created_at AS registration_date
12 FROM users u
13 LEFT JOIN bookings b ON u.uid = b.user_id AND b.status = 'confirmed'
14 GROUP BY
15     u.uid, u.full_name, u.email, u.is_active, u.created_at;

```

Listing 30: Create a view summarizing user booking activity

Relational Algebra:

- Left join users with confirmed bookings:

$$J := \text{users} \bowtie_{\text{users.uid}=\text{bookings.user_id} \wedge \text{bookings.status}='confirmed'} \text{bookings}$$

- Group by user attributes, aggregate booking counts, sums, averages, and max date:

$$\gamma_{u.uid, u.full_name, u.email, u.is_active, u.created_at; \left\{ \begin{array}{l} \text{COUNT}(b.id) \rightarrow \text{total_bookings}, \\ \text{SUM}(b.total_amount) \rightarrow \text{total_spent}, \\ \text{AVG}(b.total_amount) \rightarrow \text{avg_spending}, \\ \text{MAX}(b.booking_date) \rightarrow \text{last_booking_date} \end{array} \right.} (J)$$

Query Output:

	uid uuid	full_name character varying (255)	email character varying (255)	is_active boolean	total_bookings bigint	total_spent numeric	avg_spending numeric	last_booking_date timestamp without ti	registration_date timestamp without time zone
2	1f37c497-2b0a-4c20-b5f3-88a3a...	Alice Wonderland	alice@example.com	true	1	13500.00	13500.000000	2025-07-10 12:00...	2025-07-16 01:30:51.639812
3	3d65de21-cf6e-4f78-b83f-57eb2b...	Julia Roberts	julia@example.com	true	0	0	0	[null]	2025-07-16 01:30:51.639812
4	5a850a3-c6b0-4578-b6ec-70a38...	Mehedi Hasan	hasanmehedi26696@gmail.com	true	2	16100.00	8050.000000	2025-07-14 20:16...	2025-07-08 21:47:38.12295
5	5e99a3c8-19e2-4a90-8049-344cfc...	Charlie Chaplin	charlie@example.com	true	1	12500.00	12500.000000	2025-07-08 09:00...	2025-07-16 01:30:51.639812
6	77f7cbbc-464e-46de-8826-0b4c1...	Hannah Montana	hannah@example.com	true	0	0	0	[null]	2025-07-16 01:30:51.639812
7	89b01e4b-4690-4a4a-ae6e-87f11...	Ivan Drago	ivan@example.com	true	0	0	0	[null]	2025-07-16 01:30:51.639812
8	a95e7e2b-1e4a-41e9-86e1-7a598...	George Orwell	george@example.com	true	1	9800.00	9800.000000	2025-07-04 10:30...	2025-07-16 01:30:51.639812
9	b9dc1d92-d3aa-4db0-9880-91d7e...	Fiona Gallagher	fiona@example.com	true	0	0	0	[null]	2025-07-16 01:30:51.639812
10	c381a0ec-603e-482a-b52e-795d1...	Abu Bakar Siddique	bojackabs@gmail.com	true	0	0	0	[null]	2025-07-16 01:03:50.238281

5.2 Queries Using Views

5.2.1 Using package_summary View

```

1 -- Query: Find top performing packages using view
2 SELECT
3     title,
4     destination_name,
5     country,
6     total_bookings,
7     total_revenue
8 FROM package_summary
9 WHERE total_bookings > 0
10 ORDER BY total_revenue DESC, total_bookings DESC
11 LIMIT 10;

```

Listing 31: Find top performing packages using view

Relational Algebra (Conceptual from View):

Let P be the result of the package-summary view.

1. Select only packages with bookings:

$$\sigma_{\text{total_bookings} > 0}(P)$$

2. Project relevant columns:

$$\pi_{\text{title, destination_name, country, total_bookings, total_revenue}}(\dots)$$

3. Sort by total revenue and bookings (descending):

$$\tau_{\text{total_revenue DESC, total_bookings DESC}}(\dots)$$

4. Limit to top 10 results:

$$\delta_{10}(\dots)$$

Query Output:

	title character varying (255) 🔒	destination_name character varying (255) 🔒	country character varying (100) 🔒	total_bookings bigint 🔒	total_revenue numeric 🔒
1	Saint Martin Getaway	Saint Martin's	Bangladesh	2	27000.00
2	Cox's Beach Bonanza	Cox's Bazar Beach	Bangladesh	2	25000.00
3	Jaflong Nature Tour	Jaflong	Bangladesh	2	19600.00
4	Mystic Sylhet Tea Garden Retreat	Sylhet	Bangladesh	1	3600.00

5.2.2 Using user_activity_summary View

```

1 -- Query: Find high-value customers using view
2 SELECT
3     full_name,
4     email,
5     total_bookings,
6     total_spent,
7     avg_spending
8 FROM user_activity_summary
9 WHERE total_spent > 2000
10 ORDER BY total_spent DESC;

```

Listing 32: Find high-value customers using view

Relational Algebra (Conceptual from View):

Let U be the result of the user-activity-summary view.

1. Select users who spent more than 2000:

$$\sigma_{\text{total_spent} > 2000}(U)$$

2. Project relevant user activity attributes:

$$\pi_{\text{full_name, email, total_bookings, total_spent, avg_spending}}(\dots)$$

3. Sort by total spending in descending order:

$$\tau_{\text{total_spent DESC}}(\dots)$$

Query Output:

	full_name character varying (255) 🔒	email character varying (255) 🔒	total_bookings bigint 🔒	total_spent numeric 🔒	avg_spending numeric 🔒
1	Mehedi Hasan	hasanmehedi26696@gmail.com	2	16100.00	8050.0000000000000000
2	Alice Wonderland	alice@example.com	1	13500.00	13500.0000000000000000
3	Ethan Hunt	ethan@example.com	1	13500.00	13500.0000000000000000
4	Charlie Chaplin	charlie@example.com	1	12500.00	12500.0000000000000000
5	George Orwell	george@example.com	1	9800.00	9800.0000000000000000
6	Bob Builder	bob@example.com	1	9800.00	9800.0000000000000000

6 Functional Dependencies and Normalization

6.1 Functional Dependencies Analysis

This section analyzes the functional dependencies (FDs) for all nine tables in the VoyageVista database system, examining how attributes depend on primary keys and other determinants.

6.1.1 Users Table Functional Dependencies

Table Schema: users(uid, email, full_name, phone, passport, password_hash, is_active, bookings_count, created_at, updated_at)

Functional Dependencies:

- $\text{uid} \rightarrow \text{email, full_name, phone, passport, password_hash, is_active, bookings_count, created_at, updated_at}$
- $\text{email} \rightarrow \text{uid, full_name, phone, passport, password_hash, is_active, bookings_count, created_at, updated_at}$
- $\text{passport} \rightarrow \text{uid, email, full_name, phone, password_hash, is_active, bookings_count, created_at, updated_at}$

Candidate Keys: {uid}, {email}, {passport}

6.1.2 Destinations Table Functional Dependencies

Table Schema: destinations(id, name, country, city, description, featured_image, is_active, created_at, updated_at)

Functional Dependencies:

- $id \rightarrow name, country, city, description, featured_image, is_active, created_at, updated_at$
- $(country, city) \rightarrow name$ (assuming unique city names per country)

Candidate Keys: {id}

6.1.3 Packages Table Functional Dependencies

Table Schema: packages(id, title, description, price, destination_id, featured_image, is_featured, is_active, created_at, updated_at)

Functional Dependencies:

- $id \rightarrow title, description, price, destination_id, featured_image, is_featured, is_active, created_at, updated_at$
- $destination_id \nrightarrow id$ (Not a functional dependency — one destination can have multiple packages)

Candidate Keys: {id}

6.1.4 Package Detail Schedule Table Functional Dependencies

Table Schema: package_detail_schedule(id, package_id, highlights, itinerary, duration_days, created_at, updated_at)

Functional Dependencies:

- $id \rightarrow package_id, highlights, itinerary, duration_days, created_at, updated_at$
- $package_id \rightarrow id, highlights, itinerary, duration_days, created_at, updated_at$ (One-to-one relationship)

Candidate Keys: {id}, {package_id}

6.1.5 Promo Codes Table Functional Dependencies

Table Schema: promo_codes(id, code, description, discount_type, discount_value, start_date, expiry_date, is_active)

Functional Dependencies:

- $id \rightarrow code, description, discount_type, discount_value, start_date, expiry_date, is_active$
- $code \rightarrow id, description, discount_type, discount_value, start_date, expiry_date, is_active$

Candidate Keys: {id}, {code}

6.1.6 Bookings Table Functional Dependencies

Table Schema: bookings(id, package_id, user_id, promo_code_id, status, payment_status, booking_date, total_amount, paid_amount, discount_amount)

Functional Dependencies:

- $\text{id} \rightarrow \text{package_id}, \text{user_id}, \text{promo_code_id}, \text{status}, \text{payment_status}, \text{booking_date}, \text{total_amount}, \text{paid_amount}, \text{discount_amount}$
- $(\text{user_id}, \text{package_id}, \text{booking_date}) \rightarrow \text{id}$ (Composite candidate key in some business scenarios)

Candidate Keys: {id}

6.1.7 Blogs Table Functional Dependencies

Table Schema: blogs(id, title, content, featured_image, author_id, is_published, created_at, updated_at)

Functional Dependencies:

- $\text{id} \rightarrow \text{title}, \text{content}, \text{featured_image}, \text{author_id}, \text{is_published}, \text{created_at}, \text{updated_at}$
- $\text{title} \rightarrow \text{id}, \text{content}, \text{featured_image}, \text{author_id}, \text{is_published}, \text{created_at}, \text{updated_at}$ (Assuming unique titles)

Candidate Keys: {id}, {title}

6.1.8 Admins Table Functional Dependencies

Table Schema: admins(id, username, email, full_name, password_hash, is_super_admin, is_active, created_at, updated_at)

Functional Dependencies:

- $\text{id} \rightarrow \text{username}, \text{email}, \text{full_name}, \text{password_hash}, \text{is_super_admin}, \text{is_active}, \text{created_at}, \text{updated_at}$
- $\text{username} \rightarrow \text{id}, \text{email}, \text{full_name}, \text{password_hash}, \text{is_super_admin}, \text{is_active}, \text{created_at}, \text{updated_at}$
- $\text{email} \rightarrow \text{id}, \text{username}, \text{full_name}, \text{password_hash}, \text{is_super_admin}, \text{is_active}, \text{created_at}, \text{updated_at}$

Candidate Keys: {id}, {username}, {email}

6.1.9 Package Images Table Functional Dependencies

Table Schema: package_images(id, package_id, image_url, alt_text, is_primary, created_at)

Functional Dependencies:

- $\text{id} \rightarrow \text{package_id}, \text{image_url}, \text{alt_text}, \text{is_primary}, \text{created_at}$
- $(\text{package_id}, \text{is_primary}) \rightarrow \text{id}, \text{image_url}, \text{alt_text}, \text{created_at}$ (Only one primary image per package)

Candidate Keys: {id}

6.2 Normalization Analysis

6.2.1 First Normal Form (1NF)

Definition: A table is in 1NF if:

- All attributes contain atomic (indivisible) values
- No repeating groups exist
- Each row is uniquely identifiable

Analysis - All Tables Compliance:

```

1 -- VIOLATES 1NF (Before normalization)
2 CREATE TABLE users_unnormalized (
3     uid uuid,
4     email varchar(255),
5     full_name varchar(255),
6     phone_numbers varchar(500), -- "123-456-7890, 987-654-3210"
7     addresses text,             -- "123 Main St, City A; 456 Oak Ave,
                                   City B"
8     skills text                 -- "Java, Python, JavaScript"
9 );
10
11 -- COMPLIES WITH 1NF (Current design)
12 CREATE TABLE users (
13     uid uuid PRIMARY KEY,
14     email varchar(255) UNIQUE NOT NULL,
15     full_name varchar(255) NOT NULL,
16     phone varchar(20),          -- Single atomic phone value
17     passport varchar(255) UNIQUE NOT NULL,
18     password_hash varchar(255) NOT NULL,
19     is_active boolean NOT NULL DEFAULT true,
20     bookings_count integer DEFAULT 0,
21     created_at timestamp DEFAULT CURRENT_TIMESTAMP,
22     updated_at timestamp DEFAULT CURRENT_TIMESTAMP
23 );

```

Listing 33: 1NF Compliance Example - Users Table

1NF Status: All nine tables comply with 1NF as each attribute contains atomic values.

6.2.2 Second Normal Form (2NF)

Definition: A table is in 2NF if:

- It is in 1NF
- No partial dependencies exist (non-key attributes depend on entire primary key)

Analysis - Composite Key Scenarios:

```

1 -- VIOLATES 2NF (Hypothetical composite key scenario)
2 CREATE TABLE package_images_violation (
3     package_id uuid,
4     image_sequence integer,

```

```

5   package_title varchar(255),      -- Depends only on package_id (
   partial dependency)
6   package_price numeric(10,2),    -- Depends only on package_id (
   partial dependency)
7   image_url text,                  -- Depends on both package_id,
   image_sequence
8   alt_text varchar(255),          -- Depends on both package_id,
   image_sequence
9   is_primary boolean,             -- Depends on both package_id,
   image_sequence
10  PRIMARY KEY (package_id, image_sequence)
11 );
12
13 -- COMPLIES WITH 2NF (Current design)
14 CREATE TABLE packages (
15     id uuid PRIMARY KEY,
16     title varchar(255) NOT NULL,   -- Depends only on package id
17     price numeric(10,2) NOT NULL  -- Depends only on package id
18 );
19
20 CREATE TABLE package_images (
21     id uuid PRIMARY KEY,           -- Single attribute primary key
22     package_id uuid NOT NULL,      -- References packages.id
23     image_url text NOT NULL,       -- Depends on image id
24     alt_text varchar(255),         -- Depends on image id
25     is_primary boolean DEFAULT false, -- Depends on image id
26     FOREIGN KEY (package_id) REFERENCES packages(id)
27 );

```

Listing 34: 2NF Analysis - Package Images with Composite Keys

2NF Status: All tables comply with 2NF because we use single-attribute UUID primary keys, eliminating partial dependency issues.

6.2.3 Third Normal Form (3NF)

Definition: A table is in 3NF if:

- It is in 2NF
- No transitive dependencies exist (non-key attributes don't depend on other non-key attributes)

Analysis - Transitive Dependency Elimination:

```

1  -- VIOLATES 3NF (Before normalization)
2  CREATE TABLE bookings_unnormalized (
3      id uuid PRIMARY KEY,
4      package_id uuid,
5      user_id uuid,
6      package_title varchar(255),    -- Transitivity depends on
   package_id
7      package_price numeric(10,2),   -- Transitivity depends on
   package_id
8      destination_name varchar(255), -- Transitivity depends on
   package_id destination_id
9      user_email varchar(255),       -- Transitivity depends on user_id
10     user_full_name varchar(255),    -- Transitivity depends on user_id

```

```

11     promo_code varchar(50),           -- Transitively depends on
    promo_code_id
12     discount_percentage double,       -- Transitively depends on
    promo_code_id
13     booking_date timestamp,
14     total_amount numeric(10,2),
15     status varchar(20)
16 );
17
18 -- COMPLIES WITH 3NF (Current normalized design)
19 CREATE TABLE bookings (
20     id uuid PRIMARY KEY,
21     package_id uuid NOT NULL,         -- References packages.id
22     user_id uuid NOT NULL,           -- References users.uid
23     promo_code_id uuid,              -- References promo_codes.id
24     status varchar(20) NOT NULL DEFAULT 'pending',
25     payment_status varchar(20) NOT NULL DEFAULT 'pending',
26     booking_date timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
27     total_amount numeric(10,2) NOT NULL,
28     paid_amount numeric(10,2) DEFAULT 0,
29     discount_amount numeric(10,2) DEFAULT 0,
30     FOREIGN KEY (package_id) REFERENCES packages(id),
31     FOREIGN KEY (user_id) REFERENCES users(uid),
32     FOREIGN KEY (promo_code_id) REFERENCES promo_codes(id)
33 );
34
35 -- Related information stored in separate normalized tables
36 CREATE TABLE packages (
37     id uuid PRIMARY KEY,
38     title varchar(255) NOT NULL,
39     price numeric(10,2) NOT NULL,
40     destination_id uuid NOT NULL,
41     FOREIGN KEY (destination_id) REFERENCES destinations(id)
42 );
43
44 CREATE TABLE destinations (
45     id uuid PRIMARY KEY,
46     name varchar(255) NOT NULL,
47     country varchar(100) NOT NULL,
48     city varchar(100) NOT NULL
49 );
50
51 CREATE TABLE users (
52     uid uuid PRIMARY KEY,
53     email varchar(255) UNIQUE NOT NULL,
54     full_name varchar(255) NOT NULL
55 );
56
57 CREATE TABLE promo_codes (
58     id uuid PRIMARY KEY,
59     code varchar(50) UNIQUE NOT NULL,
60     discount_type varchar(20) NOT NULL,
61     discount_value double precision NOT NULL
62 );

```

Listing 35: 3NF Analysis - Bookings Table

3NF Status: All tables achieve 3NF by storing related information in separate

tables and using foreign key references, eliminating transitive dependencies.

6.2.4 Boyce-Codd Normal Form (BCNF)

Definition: A relation is in BCNF if:

- It is in 3NF
- For every non-trivial functional dependency $X \rightarrow Y$, X is a superkey

Superkey Definition: A superkey is a set of attributes that can uniquely identify each tuple in a relation. Every candidate key is a superkey, but not every superkey is a candidate key.

Analysis - BCNF Compliance for All Tables:

```

1  -- POTENTIAL BCNF VIOLATION (Hypothetical scenario)
2  CREATE TABLE blogs_violation (
3      id uuid PRIMARY KEY,
4      title varchar(255) UNIQUE,
5      content text,
6      author_username varchar(50),      -- Not a superkey
7      author_email varchar(255),        -- Depends on author_username
8      author_full_name varchar(255),    -- Depends on author_username
9      is_published boolean
10 );
11 -- Here: author_username      author_email, author_full_name (violates
12 -- BCNF)
13 -- author_username is not a superkey since it cannot uniquely identify
14 -- tuples
15
16 -- COMPLIES WITH BCNF (Current design)
17 CREATE TABLE blogs (
18     id uuid PRIMARY KEY,
19     title varchar(255) UNIQUE NOT NULL,
20     content text NOT NULL,
21     featured_image text,
22     author_id uuid NOT NULL,            -- References admins.id (superkey)
23     is_published boolean DEFAULT false,
24     created_at timestamp DEFAULT CURRENT_TIMESTAMP,
25     updated_at timestamp DEFAULT CURRENT_TIMESTAMP,
26     FOREIGN KEY (author_id) REFERENCES admins(id)
27 );
28
29 CREATE TABLE admins (
30     id uuid PRIMARY KEY,                -- Superkey
31     username varchar(50) UNIQUE,        -- Superkey
32     email varchar(255) UNIQUE,          -- Superkey
33     full_name varchar(255) NOT NULL,
34     password_hash varchar(255) NOT NULL,
35     is_super_admin boolean DEFAULT false,
36     is_active boolean DEFAULT true
37 );

```

Listing 36: BCNF Analysis - Superkey-Based Review

BCNF Compliance Summary (Superkey-Based Analysis):

1. **Users Table:** BCNF Compliant

- **Superkeys:** {uid}, {email}, {passport}, and their supersets
- All non-trivial functional dependencies have a superkey as the determinant
- Example: uid $\rightarrow$ full_name, email, phone (uid is a superkey)

2. **Destinations Table:** BCNF Compliant

- **Superkey:** {id}
- All attributes depend only on the superkey
- Example: id $\rightarrow$ name, country, city (id is a superkey)

3. **Packages Table:** BCNF Compliant

- **Superkey:** {id}
- No non-superkey determines any other attribute
- Example: id $\rightarrow$ title, price, destination_id (id is a superkey)

4. **Package Detail Schedule Table:** BCNF Compliant

- **Superkeys:** {id}, {package_id} (if one-to-one relationship)
- All dependencies are on superkeys; no partial or transitive dependency
- Example: id $\rightarrow$ highlights, itinerary (id is a superkey)

5. **Promo Codes Table:** BCNF Compliant

- **Superkeys:** {id}, {code}
- Every functional dependency has a superkey as the determinant
- Example: code $\rightarrow$ discount_type, discount_value (code is a superkey)

6. **Bookings Table:** BCNF Compliant

- **Superkey:** {id}
- All attributes are functionally dependent only on a superkey
- Example: id $\rightarrow$ user_id, package_id, total_amount (id is a superkey)

7. **Blogs Table:** BCNF Compliant

- **Superkeys:** {id}, {title}
- All dependencies originate from superkeys
- Example: title $\rightarrow$ content, author_id (title is a superkey)

8. **Admins Table:** BCNF Compliant

- **Superkeys:** {id}, {username}, {email}
- No functional dependency violates the BCNF condition
- Example: username $\rightarrow$ full_name, password_hash (username is a superkey)

9. **Package Images Table:** BCNF Compliant

- **Superkey:** {id}
- Foreign key to package is not a violating dependency; id determines all
- Example: id $\rightarrow$ package_id, image_url, alt_text (id is a superkey)

7 Frontend Design and Implementation

7.1 Frontend Architecture

7.1.1 Technology Stack

The frontend implements a modern technology stack:

- **Framework:** Next.js 15 with App Router
- **Language:** TypeScript for type safety
- **Styling:** Tailwind CSS 4 with custom design system
- **UI Components:** ShadCN UI with Radix UI primitives
- **State Management:** Zustand for global state
- **API Integration:** Custom API client with error handling
- **Authentication:** JWT with role-based access control

7.2 Frontend Architecture Advantages

7.2.1 Advantages

- **Type Safety:** TypeScript prevents runtime errors
- **Performance:** Next.js optimizations and code splitting
- **SEO:** Server-side rendering capabilities
- **Maintainability:** Component-based architecture
- **User Experience:** Responsive and intuitive interface
- **Developer Experience:** Hot reload and modern tooling

7.2.2 Disadvantages

- **Complexity:** Steep learning curve for modern stack
- **Bundle Size:** React and Next.js add overhead
- **Build Time:** TypeScript compilation increases build duration
- **Dependency Management:** Multiple dependencies require maintenance

7.3 Key Features Implementation

7.3.1 User Management

- Secure registration and login processes
- Profile management with real-time updates
- Password reset and account verification
- Role-based dashboard access

7.3.2 Package Management

- Advanced search and filtering capabilities
- Interactive package browsing with image galleries
- Real-time availability and pricing updates
- Responsive card-based layout

7.3.3 Booking System

- Multi-step booking process with validation
- Promo code integration and validation
- Payment status tracking and notifications
- Booking history and management

7.3.4 Admin Dashboard

- Comprehensive analytics and reporting
- CRUD operations for all entities
- Real-time data visualization
- User and content management tools

7.4 API Integration

7.4.1 API Endpoints

The backend exposes several RESTful API endpoints for frontend integration:

Endpoint	Method	Description
/api/auth/login	POST	User authentication and JWT token generation
/api/auth/register	POST	New user registration with validation

Endpoint	Method	Description
/api/auth/refresh	POST	JWT token refresh for session management
/api/packages	GET	Retrieve all active travel packages
/api/packages/{id}	GET	Get specific package details by ID
/api/packages/search	POST	Advanced package search with filters
/api/packages/featured	GET	Get featured travel packages
/api/destinations	GET	List all available destinations
/api/destinations/{id}	GET	Get destination details and packages
/api/destinations/popular	GET	Retrieve popular destinations
/api/bookings	GET	User booking history and status
/api/bookings	POST	Create new booking with validation
/api/bookings/{id}	PUT	Update booking status or details
/api/bookings/{id}/cancel	POST	Cancel existing booking
/api/users/profile	GET	Get user profile information
/api/users/profile	PUT	Update user profile data
/api/users/bookings	GET	User-specific booking history
/api/promo-codes/validate	POST	Validate promotional codes
/api/blogs	GET	Retrieve travel blogs and articles
/api/blogs/{id}	GET	Get specific blog content
/api/admin/dashboard	GET	Administrative analytics data
/api/admin/packages	POST, PUT, DELETE	Package management operations
/api/admin/users	GET	User management and statistics
/api/admin/bookings	GET	Booking management and reports

7.5 Website Snapshots

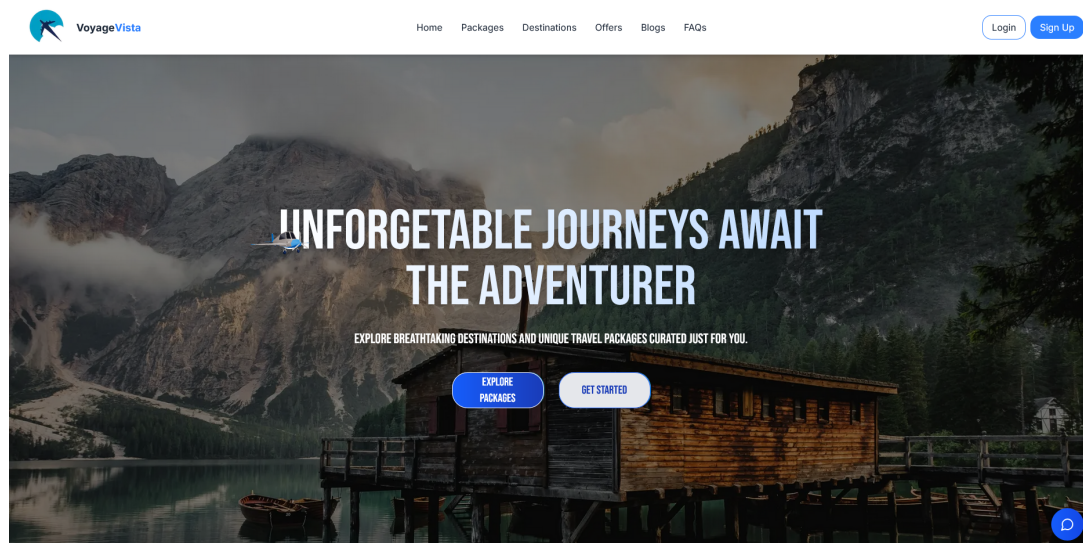


Figure 3: Landing Page

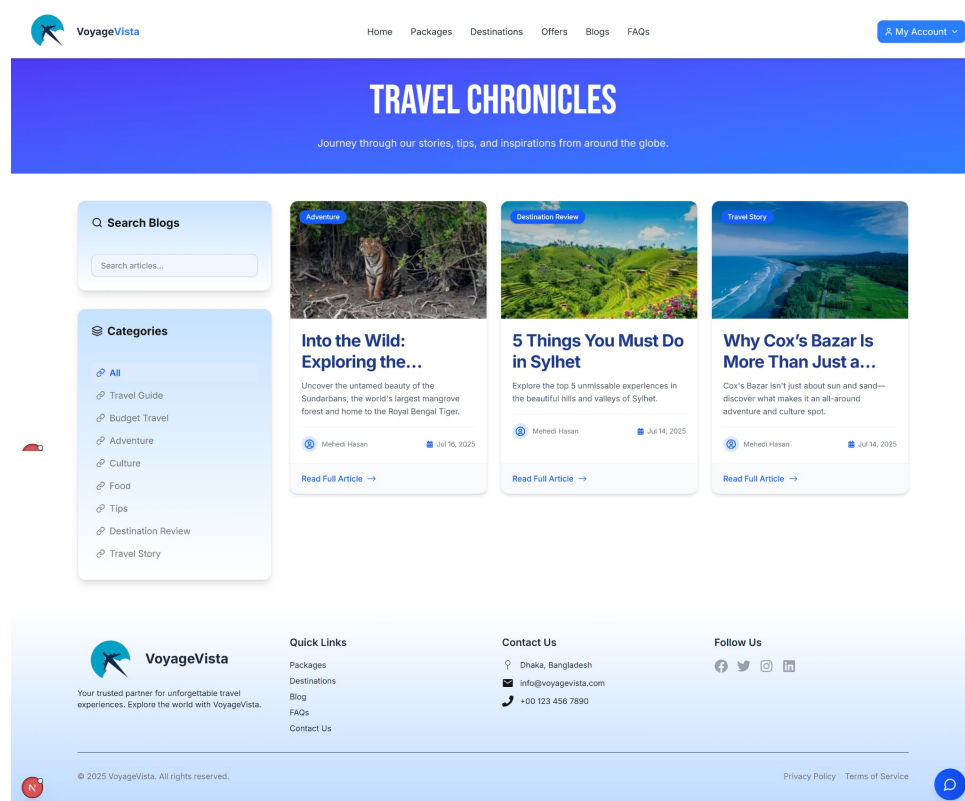


Figure 4: Blogs Page

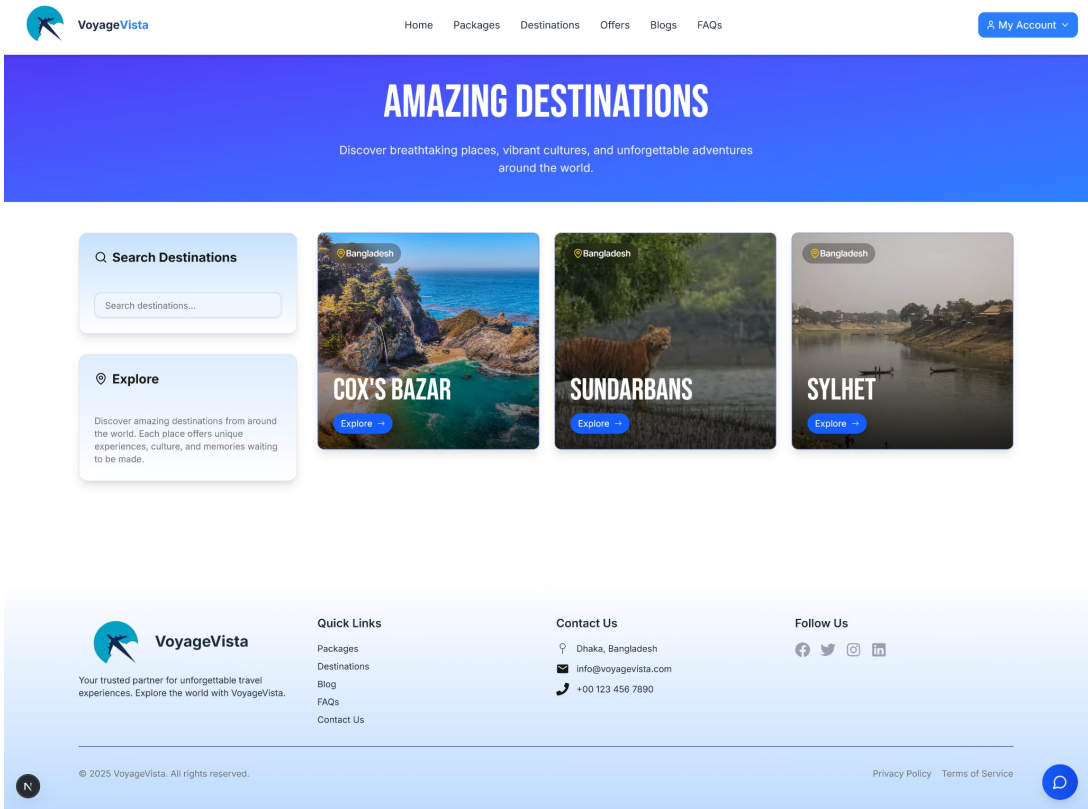


Figure 5: Destinations Page

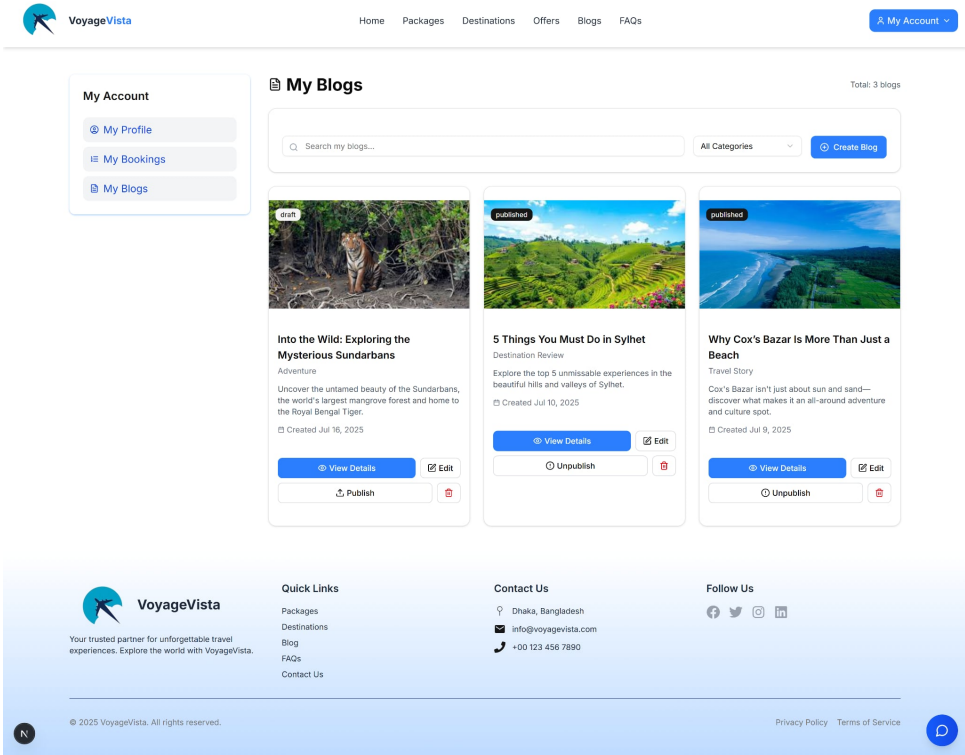


Figure 6: My Blogs Page

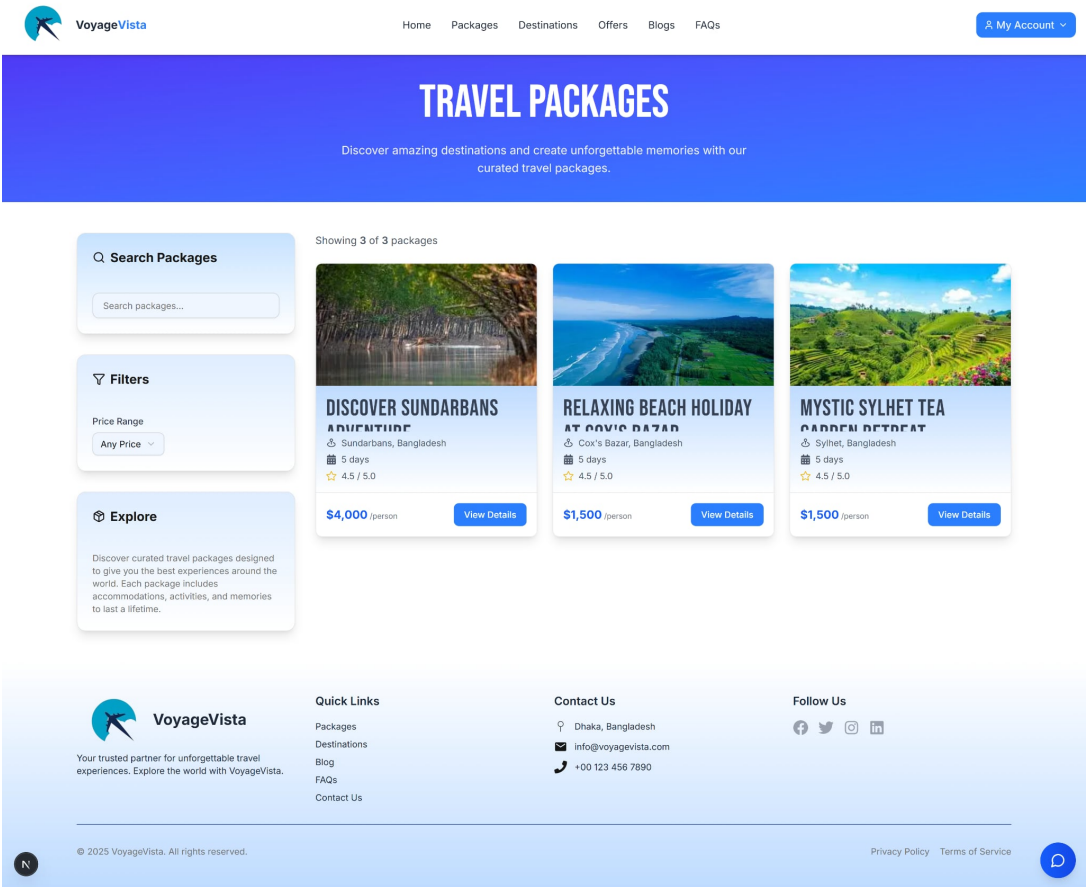


Figure 7: Packages Page

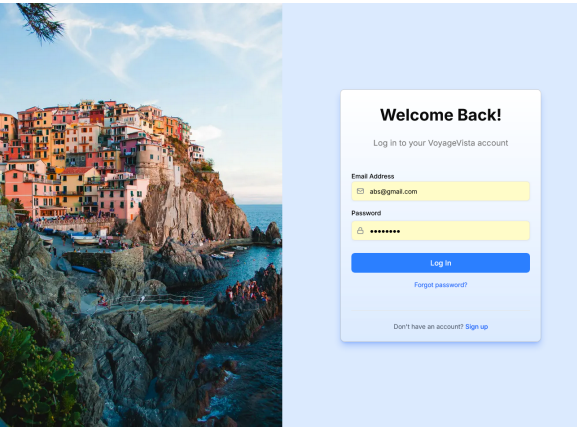


Figure 8: Login Page

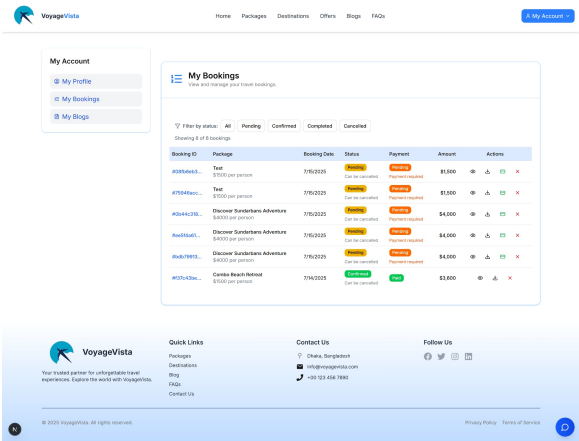


Figure 9: My Bookings

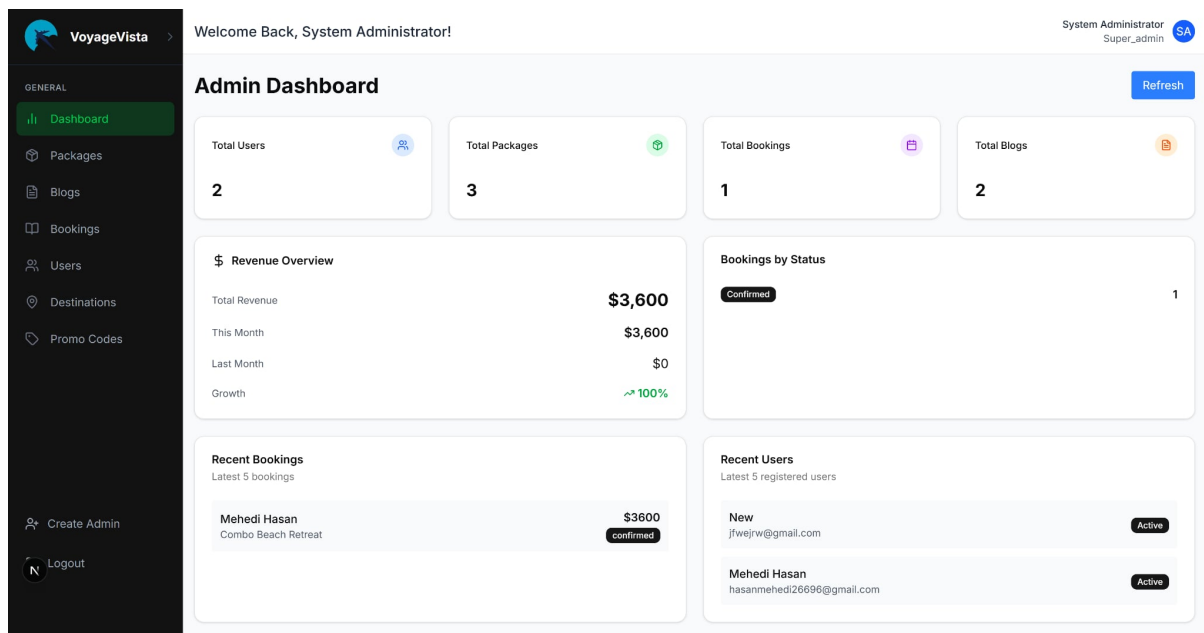


Figure 10: Admin Dashboard Page

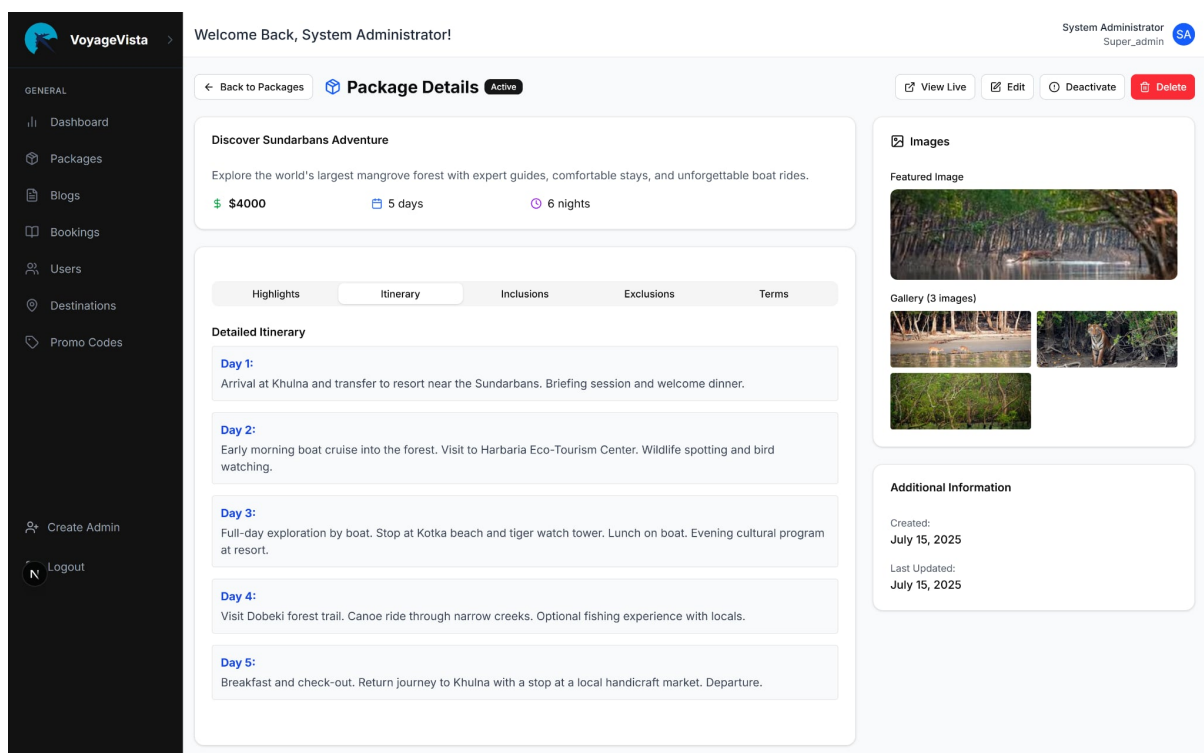


Figure 11: Admin Package Details Page

8 Conclusion

8.1 Summary

VoyageVista presents a modern, end-to-end travel booking platform that integrates a well-structured relational database with a contemporary web application. The system effectively demonstrates key concepts in database design, SQL optimization, secure authentication, and responsive frontend development. It offers a scalable and user-friendly solution that meets real-world travel management needs.

8.2 Learning Outcomes

Through this project, we gained valuable experience in:

- Database design principles and normalization techniques
- Complex SQL query formulation and optimization
- Modern web development frameworks and technologies
- System architecture and design patterns
- Frontend-backend integration and API design
- Security implementation and best practices

8.3 Future Enhancements

Potential improvements for the system include:

- Implementation of advanced analytics and machine learning
- Mobile application development
- Payment gateway integration
- Real-time notifications and communication
- Advanced search with AI-powered recommendations
- Multi-language support and internationalization